

Dynamic Programming Solutions

LeetCode Problems Grouped by Pattern

Patterns:

Linear DP 6 problems
Kadane's Algorithm 3 problems
Grid DP 6 problems
Knapsack DP 5 problems
Longest Increasing Subsequence (LIS) 3 problems
Interval DP 5 problems
String DP 8 problems
Stock Buy & Sell 6 problems
Digit DP 3 problems
Bitmask DP 2 problems
Tree DP 2 problems

Total: 49 problems

Linear DP

70. Climbing Stairs

```
package leetcode.dp.linear;

/**
 * LeetCode Problem: 70. Climbing Stairs
 * <p>
 * Question:
 * You are climbing a staircase. It takes n steps to reach the top.
 * Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?
 *
 * <p>
 * Example:
 * Input: n = 3
 * Output: 3
 * Explanation: There are three ways to climb to the top.
 * 1. 1 step + 1 step + 1 step
 * 2. 1 step + 2 steps
 * 3. 2 steps + 1 step
 *
 * <p>
 * Constraints:
 * - 1 <= n <= 45
 */
public class ClimbingStairs {

    public static int iterative(int n) {
        // Base Cases: If there are 1 or 2 stairs, the number of ways is simply n.
        if (n <= 2) {
            return n;
        }

        // dp[i] will store the number of ways to reach stair 'i'.
        int[] dp = new int[n + 1];

        // Initialize the base cases for the DP table.
        // There is 1 way to reach the 1st stair.
        dp[1] = 1;
        // There are 2 ways to reach the 2nd stair (1+1 or 2).
        dp[2] = 2;

        // Fill the DP table from the 3rd stair up to the n-th stair.
        for (int i = 3; i <= n; i++) {
            // The number of ways to reach stair 'i' is the sum of the ways to reach
            // stair 'i-1' (and taking one step) and the ways to reach stair 'i-2'
            // (and taking two steps). This is the core recurrence relation.
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        // The final answer is the number of ways to reach the n-th stair.
        return dp[n];
    }

    public static int recursive(int n, int[] dp) {
        // Base Cases: If there are 1 or 2 stairs, the number of ways is n.
        if (n <= 2) {
            return n;
        }

        // Memoization Check: If we have already computed the result for stair 'n',
        // return the stored value instead of re-calculating.
        if (dp[n] != 0) {

```

```

        return dp[n];
    }

    // Recursive Step: If the result is not in our memoization table, calculate it by
    // breaking it down into smaller subproblems.
    // The logic is the same: ways(n) = ways(n-1) + ways(n-2).
    dp[n] = recursive(n - 1, dp) + recursive(n - 2, dp);

    // Store and return the computed result.
    return dp[n];
}

/**
 * This is the main wrapper function that sets up the environment for the recursive solution.
 *
 * @param n The total number of stairs.
 * @return The number of distinct ways to climb to the top.
 */
public static int climbStairs(int n) {
    // Create a memoization array (dp table) of size n+1.
    // It's initialized to all zeros by default in Java.
    int[] dp = new int[n + 1];
    // Call the recursive helper function to start the computation.
    return recursive(n, dp);
}

/**
 * The main method to test the implemented solutions.
 */
static void main(String[] args) {
    // The climbStairs method is called, which in turn uses the recursive approach.
    // You could also call the iterative method directly for comparison.
    System.out.println("Ways to climb 2 stairs: " + climbStairs(2)); // Expected: 2
    System.out.println("Ways to climb 3 stairs: " + climbStairs(3)); // Expected: 3
    System.out.println("Ways to climb 4 stairs: " + climbStairs(4)); // Expected: 5
    System.out.println("Ways to climb 5 stairs: " + climbStairs(5)); // Expected: 8
}
}

```

91. Decode Ways

```

package leetcode.dp.linear;

/**
 * 91. Decode Ways
 * https://leetcode.com/problems/decode-ways/
 *
 * A message containing letters from A-Z can be encoded into numbers using
 * the mapping: 'A' -> "1", 'B' -> "2", ..., 'Z' -> "26".
 *
 * Given a string s containing only digits, return the number of ways to decode it.
 *
 * Example 1: Input: s = "12" -> Output: 2
 * Explanation: "12" could be decoded as "AB" (1 2) or "L" (12).
 *
 * Example 2: Input: s = "226" -> Output: 3
 * Explanation: "226" could be "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).
 *
 * Example 3: Input: s = "06" -> Output: 0
 * Explanation: "06" cannot be mapped (leading zero is invalid).
 *
 * Constraints:
 * 1 <= s.length <= 100
 * s contains only digits and may contain leading zeros.
 * ---
 */

```

```

* Approach: Linear DP (similar to Climbing Stairs with constraints)
*
* STATE:      dp[i] = number of ways to decode s[0..i-1]
* BASE:       dp[0] = 1 (empty prefix = 1 way)
* TRANSITION:
*   If s[i-1] != '0': dp[i] += dp[i-1]          (single digit decode)
*   If s[i-2..i-1] forms 10-26: dp[i] += dp[i-2] (two digit decode)
* ANSWER:     dp[n]
*
* Trace: s = "226"
*
*   dp[0] = 1   (base: empty string)
*   dp[1] = 1   s[0]='2' != '0' -> dp[0]=1
*   dp[2] = 2   s[1]='2' != '0' -> dp[1]=1; "22" in [10,26] -> dp[0]=1; total=2
*   dp[3] = 3   s[2]='6' != '0' -> dp[2]=2; "26" in [10,26] -> dp[1]=1; total=3
*
* Time:  O(n)
* Space: O(n), can optimize to O(1) with two variables
*/
public class DecodeWays {

    /**
     * Bottom-up iterative DP
     */
    public static int iterative(String s) {
        if (s == null || s.isEmpty() || s.charAt(0) == '0') return 0;

        int n = s.length();
        int[] dp = new int[n + 1];

        // Base cases
        dp[0] = 1; // Empty prefix: one way (do nothing)
        dp[1] = 1; // First character is non-zero (checked above)

        for (int i = 2; i <= n; i++) {
            // Single digit decode: s[i-1] must not be '0'
            if (s.charAt(i - 1) != '0') {
                dp[i] += dp[i - 1];
            }

            // Two digit decode: s[i-2..i-1] must be between 10 and 26
            int twoDigit = Integer.parseInt(s.substring(i - 2, i));
            if (twoDigit >= 10 && twoDigit <= 26) {
                dp[i] += dp[i - 2];
            }
        }

        return dp[n];
    }

    /**
     * Space-optimized O(1) -- only need prev two values
     */
    public static int iterativeOptimized(String s) {
        if (s == null || s.isEmpty() || s.charAt(0) == '0') return 0;

        int prevPrev = 1; // dp[i-2]
        int prev = 1;      // dp[i-1]

        for (int i = 2; i <= s.length(); i++) {
            int curr = 0;

            // Single digit
            if (s.charAt(i - 1) != '0') {
                curr += prev;
            }

            // Two digit
            int twoDigit = Integer.parseInt(s.substring(i - 2, i));
            if (twoDigit >= 10 && twoDigit <= 26) {

```

```

        curr += prevPrev;
    }

    prevPrev = prev;
    prev = curr;
}

return prev;
}

/**
 * Top-down recursive with memoization
 */
public static int recursive(String s) {
    Integer[] memo = new Integer[s.length()];
    return helper(s, 0, memo);
}

private static int helper(String s, int index, Integer[] memo) {
    // Reached the end: one valid decoding found
    if (index == s.length()) return 1;

    // Leading zero: invalid
    if (s.charAt(index) == '0') return 0;

    if (memo[index] != null) return memo[index];

    // Take one digit
    int ways = helper(s, index + 1, memo);

    // Take two digits (if valid)
    if (index + 1 < s.length()) {
        int twoDigit = Integer.parseInt(s.substring(index, index + 2));
        if (twoDigit <= 26) {
            ways += helper(s, index + 2, memo);
        }
    }

    memo[index] = ways;
    return ways;
}

public static int numDecodings(String s) {
    return iterative(s);
}

public static void main(String[] args) {
    System.out.println("\"12\": " + numDecodings("12")); // 2
    System.out.println("\"226\": " + numDecodings("226")); // 3
    System.out.println("\"06\": " + numDecodings("06")); // 0
    System.out.println("\"11106\": " + numDecodings("11106")); // 2

    System.out.println("--- Recursive ---");
    System.out.println("\"226\": " + recursive("226")); // 3

    System.out.println("--- Optimized ---");
    System.out.println("\"226\": " + iterativeOptimized("226")); // 3
}
}

```

198. House Robber

```

package leetcode.dp.linear;

/**
 * LeetCode Problem: 198. House Robber
 * <p>

```

```

* Question:
* You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed.
* The only constraint stopping you from robbing each of them is that adjacent houses have security systems connected,
* and it will automatically contact the police if two adjacent houses were broken into on the same night.
* <p>
* Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money
* you can rob tonight without alerting the police.
* <p>
* Example 1:
* Input: nums = [1, 2, 3, 1]
* Output: 4
* Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3). Total amount you can rob = 1 + 3 = 4.
* <p>
* Constraints:
* - 1 <= nums.length <= 100
* - 0 <= nums[i] <= 400
*/

```

```

public class HouseRobber {

    /**
     * Solves the problem using a recursive, top-down DP approach with memoization.
     *
     * @param nums The array of money in each house.
     * @param i The index of the current house being considered (from the end).
     * @param memo The memoization array to store results of subproblems.
     * @return The maximum amount of money that can be robbed up to house `i`.
     */
    public static int recursive(int[] nums, int i, int[] memo) {
        // Base case: If we are out of bounds (no more houses), the profit is 0.
        if (i < 0) {
            return 0;
        }

        // Memoization check: If we've already calculated the max profit for robbing up to house `i`, return it.
        if (memo[i] != -1) {
            return memo[i];
        }

        // --- The Core Choice ---
        // At house `i`, we have two options:
        // 1. Rob this house: The profit is nums[i] + the max profit from robbing houses up to `i-2` (since we can't rob
        // `i-1`).
        // 2. Don't rob this house: The profit is the max profit from robbing houses up to `i-1`.

        // We choose the option that gives us the maximum profit.
        int robThisHouse = nums[i] + recursive(nums, i - 2, memo);
        int skipThisHouse = recursive(nums, i - 1, memo);

        // Store the result in our memoization table before returning.
        memo[i] = Math.max(robThisHouse, skipThisHouse);
        return memo[i];
    }

    /**
     * Solves the problem using an iterative, bottom-up DP approach.
     *
     * @param nums The array of money in each house.
     * @return The maximum amount of money that can be robbed.
     */
    public static int iterative(int[] nums) {
        if (nums.length == 0) return 0;
        if (nums.length == 1) return nums[0];

        // dp[i] will store the maximum money that can be robbed from the first `i` houses.
        // We use n+1 size for easier indexing, where dp[0] is a dummy base case.
        int n = nums.length + 1;
        int[] dp = new int[n];

        // Base Cases:
    }

```

```

// dp[0] = 0: Max money from 0 houses is 0.
// dp[1] = nums[0]: Max money from the first house is just its own value.
dp[0] = 0;
dp[1] = nums[0];

// Build the solution from the second house onwards.
// Note: i corresponds to the house at nums[i-1].
for (int i = 2; i < n; i++) {
    // --- The Core Choice (Recurrence Relation) ---
    // The logic is the same as the recursive approach. To find the max profit for the first `i` houses (dp[i]):
    // 1. Rob house `i-1`: Profit = nums[i-1] + max profit from the first `i-2` houses (dp[i-2]).
    // 2. Don't rob house `i-1`: Profit = max profit from the first `i-1` houses (dp[i-1]).
    dp[i] = Math.max(nums[i - 1] + dp[i - 2], dp[i - 1]);
}

// The final answer is the max profit considering all houses (up to n-1).
return dp[n - 1];
}

static void main(String[] args) {
    int[] nums = {1, 2, 3, 1};
    System.out.println("Max profit for [1, 2, 3, 1]: " + iterative(nums)); // Expected: 4

    int[] nums2 = {2, 7, 9, 3, 1};
    System.out.println("Max profit for [2, 7, 9, 3, 1]: " + iterative(nums2)); // Expected: 12
}
}

```

213. House Robber II

```

package leetcode.dp.linear;

/**
 * LeetCode Problem: 213. House Robber II
 * <p>
 * Question:
 * You are a professional robber planning to rob houses along a street. All houses at this place are arranged in a circle.
 * That means the first house is the neighbor of the last one. Otherwise, the robbing rules are the same as House Robber I:
 * if two adjacent houses are broken into on the same night, the police will be contacted.
 * <p>
 * Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money
 * you can rob tonight without alerting the police.
 * <p>
 * Example 1:
 * Input: nums = [2, 3, 2]
 * Output: 3
 * Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2), because they are adjacent.
 * The best you can do is rob house 2 (money = 3).
 * <p>
 * Constraints:
 * - 1 <= nums.length <= 100
 * - 0 <= nums[i] <= 1000
 */
public class HouseRobber_2 {

    /**
     * This is a helper function that solves the original House Robber I problem for a linear array.
     * It calculates the maximum amount of money that can be robbed from a given list of houses
     * without robbing adjacent ones.
     *
     * @param nums A linear array of houses.
     * @return The maximum loot possible from this linear arrangement.
     */
    public static int helper(int[] nums) {
        // dp[i] will store the maximum money that can be robbed from the first `i` houses.
        int n = nums.length + 1;
        int[] dp = new int[n];
    }
}

```

```

// Base Cases
dp[0] = 0; // Max money from 0 houses is 0.
dp[1] = nums[0]; // Max money from the first house is its own value.

// Build the solution from the second house onwards.
for (int i = 2; i < n; i++) {
    // Recurrence Relation: The max profit for the first `i` houses is the greater of:
    // 1. Robbing the current house (nums[i-1]) + the max profit from non-adjacent previous houses (dp[i-2]).
    // 2. Not robbing the current house, which means taking the max profit from the previous set of houses (dp[i-1]).
    dp[i] = Math.max(nums[i - 1] + dp[i - 2], dp[i - 1]);
}

return dp[n - 1];
}

/**
 * Solves the House Robber II problem.
 * The core idea is to break the circular dependency by solving two separate linear subproblems.
 *
 * @param nums The circular array of money in each house.
 * @return The maximum amount of money that can be robbed.
 */
public static int rob(int[] nums) {

    // Base case: No houses to rob.
    if (nums.length == 0) {
        return 0;
    }

    // Base case: Only one house, so rob it.
    if (nums.length == 1) {
        return nums[0];
    }

    // --- The Core Logic for Circular Array ---
    // Since the first and last houses are adjacent, you cannot rob both.
    // This means any optimal solution will either:
    // 1. Include the first house but NOT the last house.
    // 2. Include the last house but NOT the first house.
    // We can solve this by running the standard House Robber algorithm on two linear subarrays:
    // - Subproblem 1: All houses EXCEPT the last one (index 0 to n-2).
    // - Subproblem 2: All houses EXCEPT the first one (index 1 to n-1).
    // The final answer is the maximum of these two results.

    // This implementation simulates the two subproblems by temporarily modifying the input array.
    // A cleaner approach might use Arrays.copyOfRange to create new subarrays.

    // --- Subproblem 1: Calculate max loot WITHOUT the first house ---
    int val0 = nums[0]; // Store the original value of the first house
    nums[0] = 0; // Temporarily set it to 0 to effectively "skip" it.
    int max1 = helper(nums); // Run the linear robber algorithm.
    nums[0] = val0; // Reset the array to its original state.

    // --- Subproblem 2: Calculate max loot WITHOUT the last house ---
    nums[nums.length - 1] = 0; // Temporarily set the last house to 0 to "skip" it.
    int max2 = helper(nums); // Run the linear robber algorithm again.

    // The overall maximum is the best result from the two scenarios.
    return Math.max(max1, max2);
}

static void main(String[] args) {
    int[] nums = {2, 3, 2};
    System.out.println("Max profit for [2, 3, 2]: " + rob(nums)); // Expected: 3

    int[] nums2 = {1, 2, 3, 1};
    System.out.println("Max profit for [1, 2, 3, 1]: " + rob(nums2)); // Expected: 4
}

```

```
}

```

746. Min Cost Climbing Stairs

```
package leetcode.dp.linear;

/**
 * LeetCode Problem: 746. Min Cost Climbing Stairs
 * <p>
 * Question:
 * You are given an integer array `cost` where `cost[i]` is the cost of the `i`-th step on a staircase.
 * Once you pay the cost, you can either climb one or two steps.
 * You can either start from the step with index 0, or the step with index 1.
 * Return the minimum cost to reach the top of the floor. The top of the floor is considered one step past the last index.
 * <p>
 * Example 1:
 * Input: cost = [10, 15, 20]
 * Output: 15
 * Explanation: You will start at index 1. Pay 15 and climb two steps to reach the top. The total cost is 15.
 * <p>
 * Example 2:
 * Input: cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]
 * Output: 6
 * Explanation: You will start at index 0. Pay 1, climb two steps to index 2. Pay 1, climb two steps to index 4. Pay 1, climb
 two steps to index 6. Pay 1, climb one step to index 7. Pay 1, climb two steps to index 9. Pay 1, climb one step to reach the
 top. The total cost is 6.
 * <p>
 * Constraints:
 * - 2 <= cost.length <= 1000
 * - 0 <= cost[i] <= 999
 */
public class MinCostClimbingStairs {

    public static int recursive(int[] cost, int[] dp, int n) {
        // Base case: If we are out of bounds, the cost is 0.
        if (n < 0) return 0;

        // Base cases: For the first two stairs (0 and 1), the cost to reach them is just their own cost.
        if (n < 2) return cost[n];

        // Memoization check: If we have already calculated the min cost for stair 'n', return it.
        if (dp[n] != 0) return dp[n];

        // Recursive Step (Recurrence Relation):
        // The minimum cost to reach stair 'n' is its own cost plus the minimum of the costs
        // to reach the previous two stairs (n-1 or n-2).
        dp[n] = cost[n] + Math.min(recursive(cost, dp, n - 1), recursive(cost, dp, n - 2));

        return dp[n];
    }

    public static int iterative(int[] cost, int[] dp, int n) {
        // Iterate through each stair to calculate the minimum cost to reach it.
        for (int i = 0; i < n; i++) {
            if (i < 2) {
                // For the first two stairs, the min cost is just their own cost,
                // as we can start from either of them.
                dp[i] = cost[i];
            } else if (i == 2){
                // Recurrence Relation: The min cost to reach stair 'i' is its own cost
                // plus the minimum of the costs to reach the previous two stairs.
                dp[i] = cost[i] + Math.min(dp[i - 1], dp[i - 2]);
            } else{
                dp[i] = cost[i] + dp[i - 1];
            }
        }
    }
}
```

```

    // The "top" of the floor is past the last step. To get there, we could have
    // come from the last step (n-1) or the second-to-last step (n-2).
    // We return the minimum of these two paths.
    return Math.min(dp[n - 1], dp[n - 2]);
}

public static int minCostClimbingStairs(int[] cost) {
    int n = cost.length;
    // The dp array will store the minimum cost to reach step 'i'.
    int[] dp = new int[n];

    // Call the iterative (bottom-up) solution.
    return iterative(cost, dp, n);
    // return Math.min(recursive(cost, dp, n - 1), recursive(cost, dp, n - 2));
}

static void main(String[] args) {
    int[] cost1 = {10, 15, 20};
    System.out.println("Min cost for [10, 15, 20]: " + minCostClimbingStairs(cost1)); // Expected: 15

    int[] cost2 = {1, 100, 1, 1, 1, 100, 1, 1, 100, 1};
    System.out.println("Min cost for the long array: " + minCostClimbingStairs(cost2)); // Expected: 6
}
}

```

279. Perfect Squares

```

package leetcode.dp.linear;

/**
 * 279. Perfect Squares
 * https://leetcode.com/problems/perfect-squares/
 *
 * Given an integer n, return the least number of perfect square numbers that
 * sum to n.
 *
 * A perfect square is an integer that is the square of an integer; in other
 * words, it is the product of some integer with itself. For example, 1, 4, 9,
 * and 16 are perfect squares while 3 and 11 are not.
 *
 * Example 1: Input: n = 12 -> Output: 3 (12 = 4 + 4 + 4)
 * Example 2: Input: n = 13 -> Output: 2 (13 = 4 + 9)
 *
 * Constraints: 1 <= n <= 10^4
 *
 * ---
 * Approach: Unbounded Knapsack / BFS
 *
 * Think of it as: we have "coins" of values 1, 4, 9, 16, 25, ...
 * and we want the minimum number of coins to make amount n.
 * This is exactly the Coin Change problem (LC 322) with square numbers as coins.
 *
 * STATE: dp[i] = minimum number of perfect squares that sum to i
 * BASE: dp[0] = 0
 * TRANSITION:
 * dp[i] = min(dp[i - j*j] + 1) for all j where j*j <= i
 *
 * Trace: n = 12
 * dp[0] = 0
 * dp[1] = dp[0] + 1 = 1 (1)
 * dp[2] = dp[1] + 1 = 2 (1+1)
 * dp[3] = dp[2] + 1 = 3 (1+1+1)
 * dp[4] = min(dp[3] + 1, dp[0] + 1) = 1 (4)
 * dp[5] = min(dp[4] + 1, dp[1] + 1) = 2 (4+1)
 * dp[6] = min(dp[5] + 1, dp[2] + 1) = 3 (4+1+1)
 * dp[7] = min(dp[6] + 1, dp[3] + 1) = 4 (4+1+1+1)

```

```

*   dp[8]  = min(dp[7]+1, dp[4]+1) = 2   (4+4)
*   dp[9]  = min(dp[8]+1, dp[5]+1, dp[0]+1) = 1   (9)
*   dp[10] = min(dp[9]+1, dp[6]+1, dp[1]+1) = 2   (9+1)
*   dp[11] = min(dp[10]+1, dp[7]+1, dp[2]+1) = 3   (9+1+1)
*   dp[12] = min(dp[11]+1, dp[8]+1, dp[3]+1) = 3   (4+4+4)   [v]
*
* Time:  O(n * sqrt(n))
* Space: O(n)
*/
public class PerfectSquares {

    // ----- Iterative DP (Unbounded Knapsack) -----
    public static int numSquares(int n) {
        int[] dp = new int[n + 1];
        java.util.Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;

        for (int i = 1; i <= n; i++) {
            // Try every perfect square j*j that fits
            for (int j = 1; j * j <= i; j++) {
                dp[i] = Math.min(dp[i], dp[i - j * j] + 1);
            }
        }

        return dp[n];
    }

    // ----- BFS (Level-order shortest path) -----
    /**
     * BFS treats the problem as a shortest path in an unweighted graph:
     * - Node 0 is the start, node n is the target.
     * - From node v, edges go to v + j*j for all valid j.
     * - BFS level = number of perfect squares used.
     */
    public static int numSquaresBFS(int n) {
        boolean[] visited = new boolean[n + 1];
        java.util.Queue<Integer> queue = new java.util.LinkedList<>();
        queue.offer(0);
        visited[0] = true;
        int level = 0;

        while (!queue.isEmpty()) {
            level++;
            int size = queue.size();
            for (int k = 0; k < size; k++) {
                int curr = queue.poll();
                // Try adding each perfect square
                for (int j = 1; curr + j * j <= n; j++) {
                    int next = curr + j * j;
                    if (next == n) return level;
                    if (!visited[next]) {
                        visited[next] = true;
                        queue.offer(next);
                    }
                }
            }
        }

        return level; // Should not reach here for valid input
    }

    public static void main(String[] args) {
        System.out.println("DP   - n=12: " + numSquares(12));           // 3
        System.out.println("BFS  - n=12: " + numSquaresBFS(12));       // 3
        System.out.println("DP   - n=13: " + numSquares(13));           // 2
        System.out.println("BFS  - n=13: " + numSquaresBFS(13));       // 2
    }
}

```

Kadane's Algorithm

152. Maximum Product Subarray

```
package leetcode.dp.kandane;

/**
 * 152. Maximum Product Subarray
 * https://leetcode.com/problems/maximum-product-subarray/
 *
 * Given an integer array nums, find a subarray that has the largest product,
 * and return the product.
 *
 * Example 1: Input: nums = [2,3,-2,4] -> Output: 6 (subarray [2,3])
 * Example 2: Input: nums = [-2,0,-1] -> Output: 0 (subarray [0])
 *
 * Constraints:
 * 1 <= nums.length <= 2 * 10^4
 * -10 <= nums[i] <= 10
 *
 * ---
 * Approach: Modified Kadane's (track both max and min product)
 *
 * Because a negative number can flip the max/min, we track both.
 * When we encounter a negative number, we swap maxProd and minProd
 * before updating, since multiplying by a negative flips their roles.
 *
 * Time: O(n)
 * Space: O(1)
 */
public class MaxProductSubarray {

    public static int maxProduct(int[] nums) {
        if (nums == null || nums.length == 0) {
            return 0;
        }

        int maxProd = nums[0]; // Maximum product ending at current position
        int minProd = nums[0]; // Minimum product ending at current position (needed for negatives)
        int result = nums[0]; // Global maximum product found so far

        for (int i = 1; i < nums.length; i++) {
            // If current number is negative, swap max and min.
            // A large negative min * negative num = large positive max.
            if (nums[i] < 0) {
                int temp = maxProd;
                maxProd = minProd;
                minProd = temp;
            }

            // Either start a new subarray at nums[i], or extend the current one
            maxProd = Math.max(nums[i], maxProd * nums[i]);
            minProd = Math.min(nums[i], minProd * nums[i]);

            // Update global result
            result = Math.max(result, maxProd);
        }

        return result;
    }

    public static void main(String[] args) {
        // Test case 1
        int[] nums1 = { 2, 3, -2, 4 };
        System.out.println(maxProduct(nums1));
    }
}
```

```

// Test case 2
int[] nums2 = { -2, 0, -1 };
System.out.println(maxProduct(nums2));
}
}

```

53. Maximum Subarray

```

package leetcode.dp.kandane;

/**
 * 53. Maximum Subarray
 * https://leetcode.com/problems/maximum-subarray/
 *
 * Given an integer array nums, find the subarray with the largest sum,
 * and return its sum.
 *
 * Example 1: Input: nums = [-2,1,-3,4,-1,2,1,-5,4] -> Output: 6
 * Explanation: The subarray [4,-1,2,1] has the largest sum 6.
 *
 * Example 2: Input: nums = [1] -> Output: 1
 * Example 3: Input: nums = [5,4,-1,7,8] -> Output: 23
 *
 * Constraints:
 * 1 <= nums.length <= 10^5
 * -10^4 <= nums[i] <= 10^4
 *
 * ---
 * Approach: Kadane's Algorithm
 *
 * Maintain a running sum. At each element, decide: extend the current
 * subarray or start fresh from the current element.
 *
 * STATE:      sum = max subarray sum ending at current index
 * TRANSITION: sum = max(sum + nums[i], nums[i])
 * ANSWER:     max of all sum values
 *
 * Time: O(n)
 * Space: O(1)
 */
public class MaxSubarray {

    public static int maxSubArray(int[] nums) {
        int max = nums[0]; // Global maximum subarray sum
        int sum = nums[0]; // Current subarray sum ending at position i

        for (int i = 1; i < nums.length; i++) {
            // Either extend the previous subarray or start a new one at nums[i]
            sum = Math.max(sum + nums[i], nums[i]);
            // Update global maximum
            max = Math.max(sum, max);
        }

        return max;
    }

    public static void main(String[] args) {
        int[] nums = {10, -5, 10};
        System.out.println(maxSubArray(nums));
    }
}

```

918. Maximum Sum Circular Subarray

```

package leetcode.dp.kandane;

/**
 * 918. Maximum Sum Circular Subarray
 * https://leetcode.com/problems/maximum-sum-circular-subarray/
 *
 * Given a circular integer array nums, find the maximum possible sum of a
 * non-empty subarray. A circular array means the end connects back to the beginning.
 *
 * Example 1: Input: nums = [1,-2,3,-2] -> Output: 3 (subarray [3])
 * Example 2: Input: nums = [5,-3,5] -> Output: 10 (subarray [5,5] wrapping around)
 * Example 3: Input: nums = [-3,-2,-3] -> Output: -2 (subarray [-2])
 *
 * Constraints:
 * n == nums.length
 * 1 <= n <= 3 * 10^4
 * -3 * 10^4 <= nums[i] <= 3 * 10^4
 *
 * ---
 * Approach: Kadane's (max + min variant)
 *
 * Two cases for the maximum subarray:
 * Case 1: Max subarray does NOT wrap around -> standard Kadane's max
 * Case 2: Max subarray WRAPS around -> totalSum - minKadane
 *         (removing the minimum middle subarray leaves the max wrapping part)
 *
 * Edge case: if maxAfterWrap == 0, all elements are negative -> use Case 1.
 *
 * Time: O(n)
 * Space: O(1)
 */
public class MaxSumCircularSubarray {

    // Standard Kadane's: find maximum subarray sum
    public static int maxKadane(int[] nums) {
        int sum = nums[0];
        int maxSum = nums[0];

        for (int i = 1; i < nums.length; i++) {
            sum = Math.max(sum + nums[i], nums[i]);
            maxSum = Math.max(maxSum, sum);
        }

        return maxSum;
    }

    // Inverted Kadane's: find minimum subarray sum
    public static int minKadane(int[] nums) {
        int minSum = nums[0];
        int sum = nums[0];

        for (int i = 1; i < nums.length; i++) {
            sum = Math.min(sum + nums[i], nums[i]);
            minSum = Math.min(minSum, sum);
        }

        return minSum;
    }

    public static int maxSubarraySumCircular(int[] nums) {
        // Case 1: max subarray is a normal (non-wrapping) subarray
        int maxKadane = maxKadane(nums);

        // Case 2: max subarray wraps around = totalSum - minSubarray
        int totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }
    }
}

```

```
int maxAfterWrap = totalSum - minKadane(nums);

// If maxAfterWrap == 0, all elements are negative; wrapping gives empty array
// In that case, return the standard Kadane result
return (maxAfterWrap == 0) ? maxKadane : Math.max(maxKadane, maxAfterWrap);
}

public static void main(String[] args) {
    int[] nums = {5, -3, 5};
    System.out.println(maxSubarraySumCircular(nums));
}
}
```

Grid DP

221. Maximal Square

```
package leetcode.dp.grid;

/**
 * 221. Maximal Square
 * https://leetcode.com/problems/maximal-square/
 *
 * Given an m x n binary matrix filled with 0's and 1's, find the largest
 * square containing only 1's and return its area.
 *
 * Example 1:
 * Input: matrix = [
 *   ["1","0","1","0","0"],
 *   ["1","0","1","1","1"],
 *   ["1","1","1","1","1"],
 *   ["1","0","0","1","0"]]
 * Output: 4 (the 2x2 square at rows 1-2, cols 2-3)
 *
 * Example 2: Input: matrix = [
 *   ["0","1"],
 *   ["1","0"]] -> Output: 1
 * Example 3: Input: matrix = [
 *   ["0"]] -> Output: 0
 *
 * Constraints:
 * m == matrix.length, n == matrix[i].length
 * 1 <= m, n <= 300
 * matrix[i][j] is '0' or '1'.
 *
 * ---
 * Approach: 2D DP
 *
 * dp[i][j] = side length of the largest square with bottom-right corner at (i, j).
 *
 * If matrix[i][j] == '1':
 *   dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
 *   (limited by the smallest of: above, left, and diagonal neighbor)
 *
 * If matrix[i][j] == '0':
 *   dp[i][j] = 0
 *
 * Example trace: matrix = [
 *   ["1","0","1","0","0"],
 *   ["1","0","1","1","1"],
 *   ["1","1","1","1","1"],
 *   ["1","0","0","1","0"]]
 *
 * dp matrix:
 *   [ 1, 0, 1, 0, 0 ]
 *   [ 1, 0, 1, 1, 1 ]
 *   [ 1, 1, 1, 2, 2 ]   <- dp[2][3]=min(1,1,1)+1=2 -> area=4
 *   [ 1, 0, 0, 1, 0 ]
 *
 * Trace for dp[2][4]:
 *   matrix[2][4]='1', min(dp[1][4], dp[2][3], dp[1][3]) = min(1,2,1) = 1
 *   dp[2][4] = 1 + 1 = 2
 *
 * ANSWER: maxSide^2
 *
 * Time: O(m * n)
 * Space: O(m * n), can optimize to O(n) using single row
 */
public class MaximalSquare {

    public static int maximalSquare(char[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
```

```

// dp[i][j] = side length of largest square ending at (i, j)
int[][] dp = new int[m][n];
int maxSide = 0;

for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (matrix[i][j] == '1') {
            if (i == 0 || j == 0) {
                // First row or first column: can only form a 1x1 square
                dp[i][j] = 1;
            } else {
                // The square is limited by the smallest neighbor
                // Think of it as: to form a kxk square at (i,j), we need
                // a (k-1)x(k-1) square at each of the three neighbors
                dp[i][j] = 1 + Math.min(dp[i - 1][j],
                    Math.min(dp[i][j - 1], dp[i - 1][j - 1]));
            }
            maxSide = Math.max(maxSide, dp[i][j]);
        }
        // If matrix[i][j] == '0', dp[i][j] stays 0
    }
}

return maxSide * maxSide; // Area = side^2
}

/**
 * Space-optimized: use only two rows instead of full m*n matrix.
 */
public static int maximalSquareOptimized(char[][] matrix) {
    int m = matrix.length;
    int n = matrix[0].length;
    int[] prev = new int[n]; // Previous row
    int[] curr = new int[n]; // Current row
    int maxSide = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == '1') {
                if (i == 0 || j == 0) {
                    curr[j] = 1;
                } else {
                    curr[j] = 1 + Math.min(prev[j],
                        Math.min(curr[j - 1], prev[j - 1]));
                }
                maxSide = Math.max(maxSide, curr[j]);
            } else {
                curr[j] = 0;
            }
        }
        // Swap rows
        int[] temp = prev;
        prev = curr;
        curr = temp;
    }

    return maxSide * maxSide;
}

public static void main(String[] args) {
    char[][] matrix1 = {
        {'1', '0', '1', '0', '0'},
        {'1', '0', '1', '1', '1'},
        {'1', '1', '1', '1', '1'},
        {'1', '0', '0', '1', '0'}
    };

    System.out.println("Example 1: " + maximalSquare(matrix1)); // 4
    System.out.println("Optimized: " + maximalSquareOptimized(matrix1)); // 4

    char[][] matrix2 = {{'0', '1'}, {'1', '0'}};

```

```

        System.out.println("Example 2: " + maximalSquare(matrix2));           // 1

        char[][] matrix3 = {{'0'}};
        System.out.println("Example 3: " + maximalSquare(matrix3));           // 0
    }
}

```

931. Minimum Falling Path Sum

```

package leetcode.dp.grid;

import java.util.Arrays;

/**
 * 931. Minimum Falling Path Sum
 * https://leetcode.com/problems/minimum-falling-path-sum/
 *
 * Given an n x n array of integers matrix, return the minimum sum of any
 * falling path through matrix.
 *
 * A falling path starts at any element in the first row and chooses the element
 * in the next row that is either directly below or diagonally left/right.
 * Specifically, the next element from position (row, col) is
 * (row+1, col-1), (row+1, col), or (row+1, col+1).
 *
 * Example 1: Input: matrix = [[2,1,3],[6,5,4],[7,8,9]] -> Output: 13
 * Explanation: Falling paths: 1->5->7 = 13, 1->4->8 = 13
 *
 * Example 2: Input: matrix = [[-19,57],[-40,-5]] -> Output: -59
 *
 * Constraints:
 * n == matrix.length == matrix[i].length
 * 1 <= n <= 100
 * -100 <= matrix[i][j] <= 100
 *
 * ---
 * Approach: DP (top-down memoization + bottom-up iterative)
 *
 * STATE: dp[row][col] = minimum falling path sum starting from (row, col).
 * BASE: dp[lastRow][col] = matrix[lastRow][col]
 * TRANSITION: dp[row][col] = matrix[row][col] + min(dp[row+1][col-1], dp[row+1][col], dp[row+1][col+1])
 * ANSWER: min(dp[0][col]) for all columns
 *
 * Time: O(n^2)
 * Space: O(n^2) for memoization, can be optimized to O(n)
 */
@SuppressWarnings("all")
public class MinimumFallingPathSum {

    // Top-down: recursively find the min falling path sum starting at (row, col)
    public static int helper(int[][] matrix, int row, int col, int[][] dp) {
        // Out of bounds -> return MAX so this path is never chosen
        if (row >= matrix.length || col >= matrix[0].length || row < 0 || col < 0) return Integer.MAX_VALUE;

        // Base case: last row, the falling path ends here
        if (row == matrix.length - 1) return matrix[row][col];

        // Memoization check
        if (dp[row][col] != Integer.MAX_VALUE) return dp[row][col];

        // Explore all three directions: down, down-left diagonal, down-right diagonal
        int down = helper(matrix, row + 1, col, dp);
        int dLeftDiagonal = helper(matrix, row + 1, col - 1, dp);
        int dRightDiagonal = helper(matrix, row + 1, col + 1, dp);

        // Pick the direction that gives the smallest sum
        int min = Math.min(down, dLeftDiagonal);

```

```

        min = Math.min(min, dRightDiagonal);

        dp[row][col] = matrix[row][col] + min;

        return dp[row][col];
    }

    public static int recursive(int[][] matrix) {
        int[][] dp = new int[matrix.length][matrix[0].length];
        for (int[] row : dp) {
            Arrays.fill(row, Integer.MAX_VALUE);
        }

        int min = Integer.MAX_VALUE;
        for (int i = 0; i < matrix[0].length; i++) {
            min = Math.min(helper(matrix, 0, i, dp), min);
        }

        return min;
    }

    // Bottom-up iterative approach
    // Uses n+2 columns (padding on both sides) to avoid boundary checks
    public static int iterative(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        // Extra 2 columns act as sentinels (INT_MAX) so we don't go out of bounds
        int[][] dp = new int[m][n + 2];

        for (int[] row : dp) {
            Arrays.fill(row, Integer.MAX_VALUE);
        }

        // Base case: copy first row (offset by 1 due to padding)
        for (int i = 0; i < n; i++) {
            dp[0][i + 1] = matrix[0][i];
        }

        // Fill row by row: each cell = matrix value + min(up-left, up, up-right)
        for (int i = 1; i < m; i++) {
            for (int j = 0; j < n; j++) {
                int up = dp[i - 1][j + 1];    // directly above
                int left = dp[i - 1][j];       // upper-left diagonal
                int right = dp[i - 1][j + 2];  // upper-right diagonal

                dp[i][j + 1] = matrix[i][j] + Math.min(up, Math.min(left, right));
            }
        }

        // Answer: minimum value in the last row
        int min = Integer.MAX_VALUE;
        for (int i = 0; i < n; i++) {
            min = Math.min(dp[matrix.length - 1][i + 1], min);
        }

        return min;
    }

    public static int minFallingPathSum(int[][] matrix) {
        return recursive(matrix);
    }

    public static void main(String[] args) {
        int[][] matrix = {{2, 1, 3}, {6, 5, 4}, {7, 8, 9}};
    }

```

```

        System.out.println(minFallingPathSum(matrix));
    }
}

```

64. Minimum Path Sum

```

package leetcode.dp.grid;

/**
 * 64. Minimum Path Sum
 * https://leetcode.com/problems/minimum-path-sum/
 *
 * Given a m x n grid filled with non-negative numbers, find a path from
 * top left to bottom right, which minimizes the sum of all numbers along
 * its path. You can only move either down or right at any point in time.
 *
 * Example 1: Input: grid = [[1,3,1],[1,5,1],[4,2,1]] -> Output: 7
 * Explanation: Path 1->3->1->1->1 minimizes the sum.
 *
 * Example 2: Input: grid = [[1,2,3],[4,5,6]] -> Output: 12
 *
 * Constraints:
 * m == grid.length, n == grid[i].length
 * 1 <= m, n <= 200
 * 0 <= grid[i][j] <= 200
 *
 * ---
 * Approach: DP (bottom-up iterative + top-down recursive)
 *
 * STATE:      dp[i][j] = minimum path sum to reach cell (i, j)
 * BASE:       dp[0][0] = grid[0][0]; first row/col are prefix sums
 * TRANSITION: dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])
 * ANSWER:     dp[m-1][n-1]
 *
 * Time: O(m * n)
 * Space: O(m * n), can be optimized to O(n)
 */
public class MinimumPathSum {

    // Bottom-up iterative DP
    public static int iterative(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        int[][] dp = new int[m][n];

        // Base Case: starting cell
        dp[0][0] = grid[0][0];

        // First row: can only come from the left
        for (int i = 1; i < n; i++) {
            dp[0][i] = grid[0][i] + dp[0][i - 1];
        }
        // First column: can only come from above
        for (int i = 1; i < m; i++) {
            dp[i][0] = grid[i][0] + dp[i - 1][0];
        }

        // Fill rest: each cell = its value + min(from above, from left)
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = grid[i][j] + Math.min(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        return dp[m - 1][n - 1];
    }
}

```

```

public static int recursive(int[][] grid, int row, int col) {
    if (row == 0 && col == 0)
        return grid[row][col];

    if (row == 0)
        /* when we reach the first row, we could only move horizontally.*/
        return grid[row][col] + recursive(grid, row, col - 1);

    if (col == 0)
        /* when we reach the first column, we could only move vertically.*/
        return grid[row][col] + recursive(grid, row - 1, col);

    /* we want the min sum path so we pick the cell with the less value */
    return grid[row][col] + Math.min(recursive(grid, row - 1, col), recursive(grid, row, col - 1));
}

public static void main(String[] args) {
    int[][] grid = {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}};
    System.out.println(iterative(grid));
    System.out.println(recursive(grid, grid.length - 1, grid[0].length - 1));
}
}

```

120. Triangle

```

package leetcode.dp.grid;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 120. Triangle
 * https://leetcode.com/problems/triangle/
 *
 * Given a triangle array, return the minimum path sum from top to bottom.
 * For each step, you may move to an adjacent number of the row below.
 * More formally, if you are on index i on the current row, you may move
 * to either index i or index i + 1 on the next row.
 *
 * Example 1: Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]] -> Output: 11
 * Explanation: The minimum path sum from top to bottom is 2 + 3 + 5 + 1 = 11.
 *
 * Example 2: Input: triangle = [[-10]] -> Output: -10
 *
 * Constraints:
 * 1 <= triangle.length <= 200
 * triangle[0].length == 1
 * triangle[i].length == triangle[i - 1].length + 1
 * -10^4 <= triangle[i][j] <= 10^4
 *
 * ---
 * Approach: DP (top-down memoization + bottom-up with O(n) space)
 *
 * STATE: dp[j] = minimum path sum from row i down to the bottom, at column j
 * BASE: dp[j] = last row values
 * TRANSITION: dp[j] = triangle[i][j] + min(dp[j], dp[j+1]) (process bottom-up)
 * ANSWER: dp[0] after processing all rows
 *
 * Time: O(n^2) where n = number of rows
 * Space: O(n) using 1D dp array
 */
@SuppressWarnings("all")
public class Triangle {

```

```

public static int minimumTotal(List<List<Integer>> triangle) {
    int[][] dp = new int[triangle.size()][triangle.size()];

    for (int i = 0; i < dp.length; i++) {
        Arrays.fill(dp[i], Integer.MAX_VALUE);
    }

    return iterative(triangle);
}

// Top-down: from (row, col), find the minimum path sum to the bottom
public static int recursive(List<List<Integer>> triangle, int row, int col, int[][] dp) {
    // Base case: reached the last row
    if (row == triangle.size() - 1) return triangle.get(row).get(col);

    if (dp[row][col] != Integer.MAX_VALUE) return dp[row][col];

    // Two choices: go straight down (col) or go diagonally (col+1)
    int nextRow = recursive(triangle, row + 1, col, dp);
    int diagonal = recursive(triangle, row + 1, col + 1, dp);

    dp[row][col] = triangle.get(row).get(col) + Math.min(nextRow, diagonal);

    return dp[row][col];
}

// Bottom-up iterative: process from last row upward, reusing a 1D array
public static int iterative(List<List<Integer>> triangle) {
    int n = triangle.size();
    int[] dp = new int[n];

    // Base case: fill dp with the last row of the triangle
    for (int i = 0; i < triangle.get(n - 1).size(); i++) {
        dp[i] = triangle.get(n - 1).get(i);
    }

    // Work upward: for each cell, pick the min of the two children below
    for (int i = n - 2; i >= 0; i--) {
        for (int j = 0; j < triangle.get(i).size(); j++) {
            dp[j] = triangle.get(i).get(j) + Math.min(dp[j], dp[j + 1]);
        }
    }

    // dp[0] holds the minimum path sum from top to bottom
    return dp[0];
}

public static void main(String[] args) {
    List<List<Integer>> triangle = new ArrayList<>();
    triangle.add(List.of(2));
    triangle.add(List.of(3, 4));
    triangle.add(List.of(6, 5, 7));
    triangle.add(List.of(4, 1, 8, 3));
    // Print the triangle
    System.out.println(minimumTotal(triangle));
}
}

```

62. Unique Paths

```
package leetcode.dp.grid;
```

```

import java.util.Arrays;

/**
 * LeetCode Problem: 62. Unique Paths
 * <p>
 * Question:
 * A robot is located at the top-left corner of a `m x n` grid.
 * The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of
the grid.
 * How many possible unique paths are there?
 * <p>
 * Example:
 * Input: m = 3, n = 7
 * Output: 28
 * <p>
 * Constraints:
 * - 1 <= m, n <= 100
 */
public class UniquePaths {

    public static int recursive(int m, int n, int[][] memo) {
        // Base case: If we are in the first row (m=1) or first column (n=1),
        // there is only one way to reach any cell from the start.
        if (m == 0 || n == 0) {
            return 1;
        }

        // Memoization check: If we've already computed the result for this cell, return it.
        if (memo[m][n] != -1) {
            return memo[m][n];
        }

        // Recurrence Relation: The number of ways to reach cell (m, n) is the sum of
        // ways to reach the cell to its left (m, n-1) and the cell above it (m-1, n).
        int fromLeft = recursive(m, n - 1, memo);
        int fromUp = recursive(m - 1, n, memo);

        // Store the result in the memoization table before returning.
        memo[m][n] = fromLeft + fromUp;
        return memo[m][n];
    }

    public static int iterative(int m, int n) {
        // dp[i][j] will store the number of unique paths to reach cell (i, j).
        int[][] dp = new int[m][n];

        // Base Cases: Initialize the first row and first column to 1.
        // There is only one way to reach any cell in the first row/column (by only moving right/down).
        for (int i = 0; i < m; i++) {
            dp[i][0] = 1;
        }
        for (int j = 0; j < n; j++) {
            dp[0][j] = 1;
        }

        // Fill the rest with the DP table.
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                // Recurrence Relation: The number of ways to reach cell (i, j) is the sum of
                // ways to reach the cell above (i-1, j) and the cell to the left (i, j-1).
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        // The final answer is in the bottom-right corner of the grid.
        return dp[m - 1][n - 1];
    }
}

```

```

public static int iterativeOptimized(int m, int n) {
    // --- The Space Optimization Insight ---
    // To calculate the values for the current row, we only need the values from the previous row.
    // We can optimize from O(m*n) space to O(n) space by only keeping track of two rows.

    // Create a 2-row DP array. We'll use the modulo operator to alternate between them.
    int[][] dp = new int[2][n];

    // Initialize the first row (which represents the base case).
    for (int j = 0; j < n; j++) {
        dp[0][j] = 1;
    }

    // Fill the DP table row by row, alternating between dp[0] and dp[1].
    for (int i = 1; i < m; i++) {
        // The current row we are filling is `i % 2`.
        // The previous row is `(i - 1) % 2`.

        // Base case for the current row (first column is always 1).
        dp[i % 2][0] = 1;
        for (int j = 1; j < n; j++) {
            // `dp[(i - 1) % 2][j]` is the cell from the previous row (move down).
            // `dp[i % 2][j - 1]` is the cell from the current row's left (move right).
            dp[i % 2][j] = dp[(i - 1) % 2][j] + dp[i % 2][j - 1];
        }
    }

    // The result is in the last cell of the row corresponding to the last row of the grid (m-1).
    return dp[(m - 1) % 2][n - 1];
}

static void main(String[] args) {
    int m = 6;
    int n = 6;
    // Using the space-optimized iterative approach for the test.
    int paths = iterativeOptimized(m, n);
    System.out.println("Number of unique paths for a " + m + "x" + n + " grid: " + paths);

    int[][] memo = new int[m][n];
    for (int[] row : memo) {
        Arrays.fill(row, -1);
    }
    int pathsRecursive = recursive(m - 1, n - 1, memo);
    System.out.println("Number of unique paths (recursive) for a " + m + "x" + n + " grid: " + pathsRecursive);
}
}

```

63. Unique Paths II

```

package leetcode.dp.grid;

import java.util.Arrays;

/**
 * LeetCode Problem: 63. Unique Paths II
 * <p>
 * Question:
 * You are given an `m x n` integer array `grid`. There is a robot on the top-left corner (i.e., `grid[0][0]`).
 * The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or
 * right at any point in time.
 * An obstacle and space are marked as `1` or `0` respectively in the grid. A path that the robot takes cannot include any
 * square that is an obstacle.
 * Return the number of possible unique paths that the robot can take to reach the bottom-right corner.
 * <p>
 * Example:

```

```

* Input: obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
* Output: 2
* Explanation: There is one obstacle in the middle of the 3x3 grid.
* There are two ways to reach the bottom-right corner:
* 1. Right -> Right -> Down -> Down
* 2. Down -> Down -> Right -> Right
* <p>
* Constraints:
* - m == obstacleGrid.length
* - n == obstacleGrid[i].length
* - 1 <= m, n <= 100
* - obstacleGrid[i][j] is 0 or 1.
*/

public class UniquePaths_2 {

    /**
     * Solves the problem using a recursive, top-down DP approach with memoization.
     * This function calculates the number of paths to reach cell (m, n) from (0, 0).
     *
     * @param m          The current row index.
     * @param n          The current column index.
     * @param obstacleGrid The grid with obstacles.
     * @param memo        A memoization table to store results of subproblems.
     * @return The number of unique paths to cell (m, n).
     */
    public static int recursive(int m, int n, int[][] obstacleGrid, int[][] memo) {
        // Base case: If we are out of bounds or have hit an obstacle, this path is invalid. Return 0.
        if (m < 0 || n < 0 || obstacleGrid[m][n] == 1) {
            return 0;
        }

        // Base case: If we have successfully reached the starting corner (0,0), this is one valid path.
        if (m == 0 && n == 0) {
            return 1;
        }

        // Memoization check: If we've already computed the result for this cell, return it.
        if (memo[m][n] != -1) {
            return memo[m][n];
        }

        // Recursive Step (Recurrence Relation):
        // The number of ways to reach cell (m, n) is the sum of the ways to reach the cell
        // above it (m-1, n) and the cell to its left (m, n-1).
        int fromUp = recursive(m - 1, n, obstacleGrid, memo);
        int fromLeft = recursive(m, n - 1, obstacleGrid, memo);

        // Store the result in the memoization table before returning.
        memo[m][n] = fromLeft + fromUp;
        return memo[m][n];
    }

    /**
     * Solves the problem using an iterative, bottom-up DP approach.
     * NOTE: Your original implementation had a bug in the base case handling for the first row/column.
     * This is the corrected and explained version.
     *
     * @param obstacleGrid The grid with obstacles.
     * @return The number of unique paths to the bottom-right corner.
     */
    public static int iterative(int[][] obstacleGrid) {
        int m = obstacleGrid.length;
        int n = obstacleGrid[0].length;

        // If the starting cell itself is an obstacle, no paths are possible.
        if (obstacleGrid[0][0] == 1) {
            return 0;
        }
    }
}

```

```

// dp[i][j] will store the number of unique paths to reach cell (i, j).
int[][] dp = new int[m][n];

// Base Case: There is one way to reach the starting cell.
dp[0][0] = 1;

// Fill the first column. A cell is reachable only if the cell above it was reachable
// and the current cell is not an obstacle.
for (int i = 1; i < m; i++) {
    if (obstacleGrid[i][0] == 0 && dp[i - 1][0] == 1) {
        dp[i][0] = 1;
    }
}

// Fill the first row. A cell is reachable only if the cell to its left was reachable
// and the current cell is not an obstacle.
for (int j = 1; j < n; j++) {
    if (obstacleGrid[0][j] == 0 && dp[0][j - 1] == 1) {
        dp[0][j] = 1;
    }
}

// Fill the rest of the DP table using the recurrence relation.
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        // If the current cell is an obstacle, it's unreachable.
        if (obstacleGrid[i][j] == 1) {
            dp[i][j] = 0;
        } else {
            // Otherwise, the number of paths is the sum from the cell above and the cell to the left.
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
}

// The final answer is in the bottom-right corner of the grid.
return dp[m - 1][n - 1];
}

static void main(String[] args) {
    int[][] obstacleGrid = {{0, 0, 0}, {0, 1, 0}, {0, 0, 0}};
    System.out.println("Iterative result: " + iterative(obstacleGrid)); // Expected: 2

    // Setup for the recursive solution
    int m = obstacleGrid.length;
    int n = obstacleGrid[0].length;
    int[][] memo = new int[m][n];
    for (int[] row : memo) {
        Arrays.fill(row, -1);
    }
    System.out.println("Recursive result: " + recursive(m - 1, n - 1, obstacleGrid, memo)); // Expected: 2
}
}

```

Knapsack DP

322. Coin Change

```
package leetcode.dp.knapsack;

import java.util.Arrays;

/**
 * LeetCode Problem: 322. Coin Change
 * <p>
 * Question:
 * You are given an integer array `coins` representing coins of different denominations and an integer `amount`
 * representing a total amount of money.
 * Return the fewest number of coins that you need to make up that amount. If that amount of money cannot
 * be made up by any combination of the coins, return -1.
 * You may assume that you have an infinite number of each kind of coin.
 * <p>
 * Example 1:
 * Input: coins = [1, 2, 5], amount = 11
 * Output: 3
 * Explanation: 11 = 5 + 5 + 1
 * <p>
 * Example 2:
 * Input: coins = [2], amount = 3
 * Output: -1
 * <p>
 * Constraints:
 * - 1 <= coins.length <= 12
 * - 1 <= coins[i] <= 2^31 - 1
 * - 0 <= amount <= 10^4
 */
public class CoinChange {

    public static int helper(int[] coins, int amount, int[] memo) {
        // Base case: If the amount is negative, this path is impossible.
        if (amount < 0) return -1;
        // Base case: If the amount is zero, we've successfully made the change. 0 coins needed.
        if (amount == 0) return 0;

        if (memo[amount] != -2) return memo[amount];

        // Initialize minCount to -1 (representing "impossible" for this amount so far)
        int minCount = -1;

        for (int coin : coins) {
            int count = helper(coins, amount - coin, memo);

            if (count >= 0) {
                int currentTotal = 1 + count;

                if (minCount == -1 || currentTotal < minCount) {
                    minCount = currentTotal;
                }
            }
        }

        // Store the result (either the min coins or -1) in the memo table and return it.
        memo[amount] = minCount;
        return memo[amount];
    }

    public static int recursive(int[] coins, int amount) {
        // Initialize memo table. We need `amount + 1` size to store 0 to `amount`.
        int[] memo = new int[amount + 1];
    }
}
```

```

    // Fill with a sentinel value (-2) to indicate "not yet computed".
    Arrays.fill(memo, -2);
    return helper(coins, amount, memo);
}

/**
 * Subproblem: dp[i][j] = The fewest coins to make amount j, using only the first i coin types.
 *
 * Example: If coins = {1, 2, 5} and amount = 5:
 * dp[2][5] asks: "What is the fewest coins to make 5, using only the first two coins {1, 2}?"
 * The answer would be 3 (from 1 + 2 + 2).
 */
public static int iterative(int[] coins, int amount) {
    // dp[i][j] = min coins to make amount `j` using only the first `i` coins.
    int[][] dp = new int[coins.length + 1][amount + 1];

    // Base case: 0 coins are needed to make an amount of 0.
    for (int i = 0; i <= coins.length; i++) {
        dp[i][0] = 0;
    }

    // Base case: With 0 coins, it's impossible to make any amount > 0.
    // We use `amount + 1` as a "infinity" value, since it's an unreachable number of coins.
    for (int j = 1; j <= amount; j++) {
        dp[0][j] = amount + 1;
    }

    // Fill the DP table
    for (int i = 1; i <= coins.length; i++) {
        for (int j = 1; j <= amount; j++) {
            // `coin` is the current coin we are considering
            int coin = coins[i - 1];

            if (coin <= j) {
                // We have two choices:
                // 1. Don't use the current coin: `dp[i - 1][j]`
                // 2. Use the current coin: `1 + dp[i][j - coin]` (We use `dp[i]` here, not `dp[i-1]`, because it's an
                //    Unbounded Knapsack,
                //    meaning we can use the same coin multiple times).
                dp[i][j] = Math.min(dp[i - 1][j], 1 + dp[i][j - coin]);
            } else {
                // If the coin is larger than the amount, we can't use it.
                // So, the result is the same as not having this coin.
                dp[i][j] = dp[i - 1][j];
            }
        }
    }

    // Get the final answer for all coins and the target amount.
    int result = dp[coins.length][amount];
    // If the result is still our "infinity" value, it's impossible.
    return result > amount ? -1 : result;
}

static void main(String[] args) {
    int[] coins = {1, 2, 5};
    int amount = 11;
    // Testing the most efficient (1D iterative) solution.
    System.out.println("Min coins for amount 11: " + iterative(coins, amount)); // Expected: 3

    int[] coins2 = {2};
    int amount2 = 3;
    System.out.println("Min coins for amount 3: " + iterative(coins2, amount2)); // Expected: -1
}
}

```

518. Coin Change 2

```

package leetcode.dp.knapsack;

/**
 * LeetCode Problem: 518. Coin Change 2
 * <p>
 * Question:
 * You are given an integer array `coins` representing coins of different denominations and an integer `amount`
 * representing a total amount of money.
 * Return the number of combinations that make up that amount. If that amount of money cannot be made up by
 * any combination of the coins, return 0.
 * You may assume that you have an infinite number of each kind of coin.
 * <p>
 * Example:
 * Input: amount = 5, coins = [1, 2, 5]
 * Output: 4
 * Explanation: There are four ways to make up the amount:
 * 5 = 5
 * 5 = 2 + 2 + 1
 * 5 = 2 + 1 + 1 + 1
 * 5 = 1 + 1 + 1 + 1 + 1
 * <p>
 * Constraints:
 * - 1 <= coins.length <= 300
 * - 1 <= coins[i] <= 5000
 * - 0 <= amount <= 5000
 */
public class CoinChange_2 {

    // The state is: dp[index][amount] = "Number of ways to make amount using only coins from index onwards."
    private static int helper(int[] coins, int amount, int index, Integer[][] dp) {
        // Base case: We found a valid combination.
        if (amount == 0) return 1;
        // Base case: This path is invalid.
        if (amount < 0) return 0;

        if (dp[index][amount] != null) return dp[index][amount];

        int count = 0;
        for (int i = index; i < coins.length; i++) {
            count = count + helper(coins, amount - coins[i], i, dp);
        }

        dp[index][amount] = count;
        return count;
    }

    public static int recursive(int amount, int[] coins) {
        Integer[][] memo = new Integer[coins.length][amount + 1];
        return helper(coins, amount, 0, memo);
    }

    /**
     * Subproblem: dp[i][j] = The number of combinations to make amount j, using only the first i coin types.
     *
     * Example: If coins = {1, 2, 5} and amount = 5:
     * dp[2][5] asks: "How many ways are there to make 5, using only {1, 2}?"
     * The answer would be 3 (from [1,1,1,1,1], [1,1,1,2], and [1,2,2]).
     */
    public static int Iterative(int amount, int[] coins) {
        // dp[i][j] = number of ways to make amount `j` using the first `i` coins.
        int[][] dp = new int[coins.length + 1][amount + 1];

        // Base case: There is 1 way to make amount 0 (by using no coins).
        for (int i = 0; i <= coins.length; i++) {
            dp[i][0] = 1;
        }
    }
}

```

```

// Fill the DP table
for (int i = 1; i <= coins.length; i++) {
    for (int j = 1; j <= amount; j++) {
        int coin = coins[i - 1];

        // Recurrence Relation:
        // We add two possibilities:
        // 1. Ways to make amount `j` WITHOUT using coin `i` (exclude): dp[i - 1][j]
        // 2. Ways to make amount `j` WITH coin `i` (include):
        //    This is `dp[i][j - coin]`. We look at the same row `i` because we can use the coin again.
        if (coin <= j) {
            dp[i][j] = dp[i - 1][j] + dp[i][j - coin];
        } else {
            // If the coin is larger than the amount, we can't use it.
            dp[i][j] = dp[i - 1][j];
        }
    }
}

// The final answer is in the bottom-right corner.
return dp[coins.length][amount];
}

static void main(String[] args) {
    int[] coins = {1, 2, 5};
    int amount = 5;
    System.out.println(recursive(amount, coins)); // Expected: 4
}
}

```

474. Ones and Zeroes

```

package leetcode.dp.knapsack;

/**
 * 474. Ones and Zeroes
 * https://leetcode.com/problems/ones-and-zeroes/
 *
 * You are given an array of binary strings strs and two integers m and n.
 * Return the size of the largest subset of strs such that there are at most
 * m 0's and n 1's in the subset.
 *
 * A set x is a subset of a set y if all elements of x are also elements of y.
 *
 * Example 1:
 * Input: strs = ["10", "0001", "111001", "1", "0"], m = 5, n = 3
 * Output: 4
 * Explanation: The largest subset with at most 5 zeroes and 3 ones is
 * {"10", "0001", "1", "0"}, so the answer is 4.
 * Other valid but smaller subsets: {"111001"} has 4 zeros and 2 ones.
 *
 * Example 2:
 * Input: strs = ["10", "0", "1"], m = 1, n = 1
 * Output: 2
 * Explanation: The largest subset is {"0", "1"} -> 1 zero, 1 one.
 *
 * Constraints:
 * 1 <= strs.length <= 600
 * 1 <= strs[i].length <= 100
 * strs[i] consists only of digits '0' and '1'.
 * 1 <= m, n <= 100
 *
 * ---
 * Approach: Multi-dimensional 0/1 Knapsack
 *
 * This is a classic 0/1 knapsack with TWO capacity dimensions:

```

```

* - Capacity of 0's (m) and capacity of 1's (n).
* - Each string is an "item" with cost (zeros, ones) and value 1.
*
* STATE: dp[i][j] = max subset size using at most i zeros and j ones
* BASE: dp[0][0] = 0
* TRANSITION: For each string with (zeros, ones) count:
*   dp[i][j] = max(dp[i][j], dp[i - zeros][j - ones] + 1)
*
* We iterate strings in outer loop, and iterate dp in reverse (like 0/1 knapsack)
* to ensure each string is used at most once.
*
* Time: O(L * m * n) where L = strs.length
* Space: O(m * n)
*/
public class OnesAndZeroes {

    // ----- Iterative 2D Knapsack -----
    public static int findMaxForm(String[] strs, int m, int n) {
        // dp[i][j] = max strings we can pick with at most i zeros and j ones
        int[][] dp = new int[m + 1][n + 1];

        for (String s : strs) {
            // Count zeros and ones in current string
            int zeros = 0, ones = 0;
            for (char c : s.toCharArray()) {
                if (c == '0') zeros++;
                else ones++;
            }

            // Iterate in REVERSE to avoid using the same string twice (0/1 knapsack)
            for (int i = m; i >= zeros; i--) {
                for (int j = n; j >= ones; j--) {
                    // Either skip this string or take it
                    dp[i][j] = Math.max(dp[i][j], dp[i - zeros][j - ones] + 1);
                }
            }
        }

        return dp[m][n];
    }

    // ----- Recursive + Memoization -----
    public static int findMaxFormRecursive(String[] strs, int m, int n) {
        int[][][] memo = new int[strs.length][m + 1][n + 1];
        // Initialize with -1 to distinguish "not computed" from 0
        for (int[][] layer : memo)
            for (int[] row : layer)
                java.util.Arrays.fill(row, -1);
        return solve(strs, m, n, 0, memo);
    }

    private static int solve(String[] strs, int m, int n, int idx, int[][][] memo) {
        if (idx == strs.length) return 0;
        if (memo[idx][m][n] != -1) return memo[idx][m][n];

        // Count zeros and ones
        int zeros = 0, ones = 0;
        for (char c : strs[idx].toCharArray()) {
            if (c == '0') zeros++;
            else ones++;
        }

        // Option 1: Skip current string
        int skip = solve(strs, m, n, idx + 1, memo);

        // Option 2: Take current string (if capacity allows)
        int take = 0;
        if (m >= zeros && n >= ones) {
            take = 1 + solve(strs, m - zeros, n - ones, idx + 1, memo);
        }
    }
}

```

```

        memo[idx][m][n] = Math.max(skip, take);
        return memo[idx][m][n];
    }

    public static void main(String[] args) {
        String[] strs1 = {"10", "0001", "111001", "1", "0"};
        System.out.println("Example 1: " + findMaxForm(strs1, 5, 3)); // 4
        System.out.println("Recursive: " + findMaxFormRecursive(strs1, 5, 3)); // 4

        String[] strs2 = {"10", "0", "1"};
        System.out.println("Example 2: " + findMaxForm(strs2, 1, 1)); // 2
    }
}

```

416. Partition Equal Subset Sum

```

package leetcode.dp.knapsack;

import java.util.Arrays;

/**
 * LeetCode Problem: 416. Partition Equal Subset Sum
 *
 * Question:
 * Given a non-empty array `nums` containing only positive integers, find if the array can be partitioned
 * into two subsets such that the sum of elements in both subsets is equal.
 *
 * Example 1:
 * Input: nums = [1, 5, 11, 5]
 * Output: true
 * Explanation: The array can be partitioned as [1, 5, 5] and [11]. Both have a sum of 11.
 *
 * Example 2:
 * Input: nums = [1, 2, 3, 5]
 * Output: false
 * Explanation: The array cannot be partitioned into two subsets with equal sum.
 *
 * Constraints:
 * - 1 <= nums.length <= 200
 * - 1 <= nums[i] <= 100
 */
@SuppressWarnings("all")
public class PartitionSubsetSum {

    public static boolean recursive(int[] nums, int index, int target, Boolean[][] dp) {
        // Base case: We've reached the target sum exactly.
        if (target == 0) {
            return true;
        }

        // Base cases: We've run out of numbers or overshoot the target.
        if (index > nums.length - 1 || target < 0) {
            return false;
        }

        if (dp[index][target] != null) {
            return dp[index][target];
        }

        boolean include = recursive(nums, index + 1, target - nums[index], dp);
        boolean exclude = recursive(nums, index + 1, target, dp);

        dp[index][target] = include || exclude;

        return dp[index][target];
    }
}

```

```

/**
 * Subproblem: dp[i][j] = True if sum j can be formed using a subset of the first i numbers.
 *
 * Example: If nums = {1, 5, 11, 5} and target = 11:
 * dp[2][6] asks: "Can we make a sum of 6, using a subset of the first two numbers {1, 5}?"
 * The answer would be true (from {1, 5}).
 *
 * dp[2][7] asks: "Can we make a sum of 7, using a subset of {1, 5}?"
 * The answer would be false.
 */
public static boolean iterative(int[] nums, int targetSum) {
    // dp[i][j] = can we make a sum of `j` using only the first `i` numbers?
    boolean[][] dp = new boolean[nums.length + 1][targetSum + 1];

    // Base case: We can always make a sum of 0 (by taking no elements).
    for (int i = 0; i <= nums.length; i++) {
        dp[i][0] = true;
    }

    for (int i = 1; i <= nums.length; i++) {
        int currentNum = nums[i - 1]; // The current number we are considering
        for (int j = 1; j <= targetSum; j++) {

            if (currentNum <= j) {
                // We have two choices:
                // 1. Exclude the current number: dp[i - 1][j] (result using numbers before this one)
                // 2. Include the current number: dp[i - 1][j - currentNum] (result for the remaining sum)
                dp[i][j] = dp[i - 1][j] || dp[i - 1][j - currentNum];
            } else {
                // If the number is larger than the target `j`, we can't include it.
                // So, the result is the same as not having this number.
                dp[i][j] = dp[i - 1][j];
            }
        }
    }

    // The final answer is whether we can make the `targetSum` using all `nums.length` numbers.
    return dp[nums.length][targetSum];
}

public static boolean canPartition(int[] nums) {
    int totalSum = 0;
    for (int val : nums) {
        totalSum += val;
    }

    // If the total sum is odd, it's impossible to split into two equal halves.
    if (totalSum % 2 != 0) return false;

    // The target sum for our subset is half of the total.
    int targetSum = totalSum / 2;

    // Setup for recursive solution
    Boolean dp[][] = new Boolean[nums.length][targetSum + 1];

    // Call any of the implementations. The iterative one is shown here.
    return iterative(nums, targetSum);
    // return recursive(nums, 0, targetSum, dp);
    // return iterativeOptimized(nums, targetSum);
}

public static void main(String[] args) {
    int[] nums = {1, 5, 11, 5};
    System.out.println(canPartition(nums)); // Expected: true
}
}

```

494. Target Sum

```

package leetcode.dp.knapsack;

import java.util.Arrays;

/**
 * 494. Target Sum
 * https://leetcode.com/problems/target-sum/
 *
 * You are given an integer array nums and an integer target. You want to build
 * an expression out of nums by adding one of the symbols '+' or '-' before each
 * integer in nums and then concatenate all the integers.
 *
 * Return the number of different expressions that evaluate to target.
 *
 * Example 1: Input: nums = [1,1,1,1,1], target = 3 -> Output: 5
 * Explanation: -1 + 1 + 1 + 1 + 1 = 3 (5 ways to pick which one is negative)
 *
 * Example 2: Input: nums = [1], target = 1 -> Output: 1
 *
 * Constraints:
 * 1 <= nums.length <= 20
 * 0 <= nums[i] <= 1000
 * 0 <= sum(nums[i]) <= 1000
 * -1000 <= target <= 1000
 *
 * ---
 * Approach: 0/1 Knapsack (Subset Sum reduction)
 *
 * Key insight: Partition nums into two subsets P (positive) and N (negative).
 * P - N = target
 * P + N = totalSum
 * => P = (target + totalSum) / 2
 *
 * So the problem reduces to: count subsets of nums that sum to P.
 * This is a classic 0/1 knapsack / subset sum count problem.
 *
 * Edge cases:
 * - If (target + totalSum) is odd, no valid partition exists -> return 0.
 * - If |target| > totalSum, impossible -> return 0.
 *
 * Time: O(n * P)
 * Space: O(P) with 1D optimization
 */
public class TargetSum {

    /**
     * Top-down recursive with memoization (the direct +/- approach).
     *
     * dp(index, currentSum) = number of ways to reach target from index onward.
     */
    public static int recursive(int[] nums, int target) {
        // Offset currentSum by 1000 to handle negative indices (sum range: -1000 to 1000)
        int offset = Arrays.stream(nums).sum();
        Integer[][] memo = new Integer[nums.length][2 * offset + 1];
        return helper(nums, target, 0, 0, offset, memo);
    }

    private static int helper(int[] nums, int target, int index, int currentSum, int offset, Integer[][] memo) {
        // All numbers used: check if we hit the target
        if (index == nums.length) {
            return currentSum == target ? 1 : 0;
        }

        if (memo[index][currentSum + offset] != null) {
            return memo[index][currentSum + offset];
        }

        // Two choices: add or subtract the current number

```

```

    int add = helper(nums, target, index + 1, currentSum + nums[index], offset, memo);
    int sub = helper(nums, target, index + 1, currentSum - nums[index], offset, memo);

    memo[index][currentSum + offset] = add + sub;
    return add + sub;
}

/**
 * Bottom-up 0/1 Knapsack (Subset Sum Count)
 *
 * Subproblem: dp[j] = number of subsets that sum to j
 *
 * Example trace: nums = [1,1,1,1,1], target = 3
 * totalSum = 5, P = (3+5)/2 = 4
 * Count subsets summing to 4:
 *
 * dp after init: [1, 0, 0, 0, 0]
 * after num=1: [1, 1, 0, 0, 0]
 * after num=1: [1, 2, 1, 0, 0]
 * after num=1: [1, 3, 3, 1, 0]
 * after num=1: [1, 4, 6, 4, 1]
 * after num=1: [1, 5, 10, 10, 5] -> dp[4] = 5 [v]
 */
public static int iterative(int[] nums, int target) {
    int totalSum = 0;
    for (int num : nums) totalSum += num;

    // Edge cases: impossible to partition
    if (Math.abs(target) > totalSum) return 0;
    if ((target + totalSum) % 2 != 0) return 0;

    int subsetSum = (target + totalSum) / 2;

    // 1D DP: dp[j] = number of subsets summing to j
    int[] dp = new int[subsetSum + 1];
    dp[0] = 1; // Base: empty subset sums to 0

    for (int num : nums) {
        // Traverse RIGHT to LEFT to avoid using the same element twice (0/1 knapsack)
        for (int j = subsetSum; j >= num; j--) {
            dp[j] += dp[j - num]; // Include num: ways to make (j - num) carry over
        }
    }

    return dp[subsetSum];
}

/**
 * 2D Knapsack (for clarity, same logic as iterative but unoptimized)
 *
 * dp[i][j] = number of subsets using first i numbers that sum to j
 *
 * Transition:
 * dp[i][j] = dp[i-1][j] (exclude nums[i-1])
 *           + dp[i-1][j - nums[i-1]] (include nums[i-1], if j >= nums[i-1])
 */
public static int iterative2D(int[] nums, int target) {
    int totalSum = 0;
    for (int num : nums) totalSum += num;

    if (Math.abs(target) > totalSum) return 0;
    if ((target + totalSum) % 2 != 0) return 0;

    int subsetSum = (target + totalSum) / 2;
    int n = nums.length;

    int[][] dp = new int[n + 1][subsetSum + 1];
    dp[0][0] = 1;

    for (int i = 1; i <= n; i++) {

```

```
        int num = nums[i - 1];
        for (int j = 0; j <= subsetSum; j++) {
            // Exclude current number
            dp[i][j] = dp[i - 1][j];
            // Include current number (if it fits)
            if (j >= num) {
                dp[i][j] += dp[i - 1][j - num];
            }
        }
    }

    return dp[n][subsetSum];
}

public static int findTargetSumWays(int[] nums, int target) {
    return iterative(nums, target);
}

public static void main(String[] args) {
    int[] nums1 = {1, 1, 1, 1, 1};
    System.out.println("Recursive: " + recursive(nums1, 3)); // 5
    System.out.println("Iterative: " + iterative(nums1, 3)); // 5
    System.out.println("2D: " + iterative2D(nums1, 3)); // 5

    int[] nums2 = {1};
    System.out.println("Recursive: " + recursive(nums2, 1)); // 1
    System.out.println("Iterative: " + iterative(nums2, 1)); // 1
}
}
```

Longest Increasing Subsequence (LIS)

300. Longest Increasing Subsequence

```
package leetcode.dp.lis;

import java.util.Arrays;

/**
 * LeetCode Problem: 300. Longest Increasing Subsequence
 *
 * Question:
 * Given an integer array `nums`, return the length of the longest strictly increasing subsequence.
 * A subsequence is a sequence that can be derived from an array by deleting some or no elements without changing the order
 * of the remaining elements.
 *
 * Example 1:
 * Input: nums = [10, 9, 2, 5, 3, 7, 101, 18]
 * Output: 4
 * Explanation: The longest increasing subsequence is [2, 3, 7, 101], therefore the length is 4.
 *
 * Constraints:
 * - 1 <= nums.length <= 2500
 * - -10^4 <= nums[i] <= 10^4
 */
public class LIS {

    // Subproblem: dp[i] = The length of the longest increasing subsequence that *ends* at index `i`.
    public static int iterative(int[] nums) {
        int n = nums.length;
        if (n == 0) return 0;

        int[] dp = new int[n];
        // Base case: Every element by itself is an increasing subsequence of length 1.
        Arrays.fill(dp, 1);

        int maxLIS = 1;

        // Build the solution from left to right.
        for (int i = 1; i < n; i++) {
            // For each element `i`, check all previous elements `j`.
            for (int j = 0; j < i; j++) {
                // If `nums[i]` can extend the subsequence ending at `j`...
                if (nums[i] > nums[j]) {
                    // then the new LIS length ending at `i` could be `dp[j] + 1`.
                    // We take the maximum of what we already had for `dp[i]` and this new possibility.
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
            maxLIS = Math.max(maxLIS, dp[i]);
        }

        return maxLIS;
    }

    static void main(String[] args) {
        int[] nums = {10, 9, 2, 5, 3, 7, 101, 18};
        System.out.println("Iterative (O(n^2)): " + iterative(nums)); // Expected: 4
    }
}
```

673. Number of Longest Increasing Subsequence

```

package leetcode.dp.lis;

import java.util.Arrays;

/**
 * 673. Number of Longest Increasing Subsequence
 * https://leetcode.com/problems/number-of-longest-increasing-subsequence/
 *
 * Given an integer array nums, return the number of longest increasing subsequences.
 * Notice that the sequence has to be strictly increasing.
 *
 * Example 1: Input: nums = [1,3,5,4,7] -> Output: 2
 * Explanation: The two longest increasing subsequences are [1,3,5,7] and [1,3,4,7].
 *
 * Example 2: Input: nums = [2,2,2,2,2] -> Output: 5
 * Explanation: Length of LIS is 1, and there are 5 such subsequences.
 *
 * Constraints:
 * 1 <= nums.length <= 2000
 * -10^6 <= nums[i] <= 10^6
 *
 * ---
 * Approach: DP with two arrays (length + count)
 *
 * Extend the classic LIS DP with a second array to track HOW MANY
 * subsequences achieve each length.
 *
 * STATE:
 * dp[i] = length of the longest increasing subsequence ending at index i
 * count[i] = number of LIS of that length ending at index i
 *
 * BASE: dp[i] = 1, count[i] = 1 (each element alone)
 *
 * TRANSITION: For each j < i where nums[j] < nums[i]:
 * If dp[j] + 1 > dp[i]: found a longer subsequence -> dp[i] = dp[j]+1, count[i] = count[j]
 * If dp[j] + 1 == dp[i]: found another way to reach same length -> count[i] += count[j]
 *
 * ANSWER: Sum of count[i] for all i where dp[i] == maxLength
 *
 * Example trace: nums = [1, 3, 5, 4, 7]
 *
 * dp: [1, 2, 3, 3, 4]
 * count: [1, 1, 1, 1, 2]
 *
 * ^-- 2 because both dp[2]=3 and dp[3]=3 extend to dp[4]=4
 *
 * Time: O(n^2)
 * Space: O(n)
 */
public class NumberOfLIS {

    public static int findNumberOfLIS(int[] nums) {
        int n = nums.length;
        if (n == 0) return 0;

        int[] dp = new int[n]; // Length of LIS ending at i
        int[] count = new int[n]; // Number of LIS of length dp[i] ending at i

        // Base case: each element alone forms a subsequence of length 1
        Arrays.fill(dp, 1);
        Arrays.fill(count, 1);

        int maxLen = 1;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    if (dp[j] + 1 > dp[i]) {
                        // Found a strictly longer subsequence ending at i

```

```

        dp[i] = dp[j] + 1;
        count[i] = count[j]; // Reset count: inherit j's count
    } else if (dp[j] + 1 == dp[i]) {
        // Found another way to reach the same length
        count[i] += count[j]; // Accumulate count
    }
}
}
maxLen = Math.max(maxLen, dp[i]);
}

// Sum up counts of all positions whose LIS length equals maxLen
int result = 0;
for (int i = 0; i < n; i++) {
    if (dp[i] == maxLen) {
        result += count[i];
    }
}

return result;
}

public static void main(String[] args) {
    int[] nums1 = {1, 3, 5, 4, 7};
    System.out.println("[1,3,5,4,7]: " + findNumberOfLIS(nums1)); // 2

    int[] nums2 = {2, 2, 2, 2, 2};
    System.out.println("[2,2,2,2,2]: " + findNumberOfLIS(nums2)); // 5

    int[] nums3 = {1, 2, 4, 3, 5, 4, 7, 2};
    System.out.println("[1,2,4,3,5,4,7,2]: " + findNumberOfLIS(nums3)); // 3
}
}

```

354. Russian Doll Envelopes

```

package leetcode.dp.lis;

import java.util.Arrays;

/**
 * 354. Russian Doll Envelopes
 * https://leetcode.com/problems/russian-doll-envelopes/
 *
 * You are given a 2D array of integers envelopes where envelopes[i] = [wi, hi]
 * represents the width and the height of an envelope.
 *
 * One envelope can fit into another if and only if both the width and height
 * of one envelope are strictly greater than the other envelope's width and height.
 *
 * Return the maximum number of envelopes you can Russian doll (put one inside the other).
 * You cannot rotate an envelope.
 *
 * Example 1: Input: envelopes = [[5,4],[6,4],[6,7],[2,3]] -> Output: 3
 * Explanation: [2,3] => [5,4] => [6,7]
 *
 * Example 2: Input: envelopes = [[1,1],[1,1],[1,1]] -> Output: 1
 *
 * Constraints:
 * 1 <= envelopes.length <= 10^5
 * envelopes[i].length == 2
 * 1 <= wi, hi <= 10^5
 *
 * ---
 * Approach: Sort + LIS with Binary Search (O(n log n))
 *
 * Key insight: reduce to 1D LIS problem.
 */

```

```

*
* 1. Sort envelopes by width ascending. If widths are equal, sort by height
*    DESCENDING. This ensures that for the same width, we never pick two
*    envelopes (since heights are decreasing, LIS won't select both).
*
* 2. Extract the heights array and find the LIS length using patience-sorting
*    (binary search approach).
*
* Example: [[5,4],[6,4],[6,7],[2,3]]
* After sort: [[2,3],[5,4],[6,7],[6,4]] (width asc, height desc for same width)
* Heights: [3, 4, 7, 4]
* LIS of heights: [3, 4, 7] -> length 3
*
* Time: O(n log n)
* Space: O(n)
*/
public class RussianDollEnvelopes {

    /**
     * O(n log n) approach: Sort + LIS via binary search (patience sorting)
     */
    public static int maxEnvelopes(int[][] envelopes) {
        // Sort: width ascending, height DESCENDING for same width
        // Descending height for same width prevents picking two envelopes
        // with same width in the LIS
        Arrays.sort(envelopes, (a, b) -> {
            if (a[0] != b[0]) return a[0] - b[0]; // width ascending
            return b[1] - a[1]; // height descending
        });

        // Now find LIS on heights using patience sorting
        // tails[i] = smallest possible tail element of an increasing subsequence of length i+1
        int[] tails = new int[envelopes.length];
        int size = 0; // Length of the longest subsequence found

        for (int[] env : envelopes) {
            int h = env[1];

            // Binary search: find the first element in tails >= h
            int lo = 0, hi = size;
            while (lo < hi) {
                int mid = lo + (hi - lo) / 2;
                if (tails[mid] < h) {
                    lo = mid + 1; // Look for a larger tail
                } else {
                    hi = mid; // tails[mid] >= h, try to go smaller
                }
            }

            // lo is the insertion point
            tails[lo] = h;

            // If lo == size, we extended the longest subsequence
            if (lo == size) size++;
        }

        return size;
    }

    /**
     * O(n^2) approach: Sort + classic LIS DP
     * (TLE for large inputs but easier to understand)
     */
    public static int maxEnvelopesDP(int[][] envelopes) {
        // Sort by width ascending, then height ascending
        Arrays.sort(envelopes, (a, b) -> {
            if (a[0] != b[0]) return a[0] - b[0];
            return a[1] - b[1];
        });
    }
}

```

```
int n = envelopes.length;
int[] dp = new int[n]; // dp[i] = max envelopes in chain ending at i
Arrays.fill(dp, 1);

int maxLen = 1;

for (int i = 1; i < n; i++) {
    for (int j = 0; j < i; j++) {
        // Both width and height must be strictly greater
        if (envelopes[j][0] < envelopes[i][0] && envelopes[j][1] < envelopes[i][1]) {
            dp[i] = Math.max(dp[i], dp[j] + 1);
        }
    }
    maxLen = Math.max(maxLen, dp[i]);
}

return maxLen;
}

public static void main(String[] args) {
    int[][] env1 = {{5, 4}, {6, 4}, {6, 7}, {2, 3}};
    System.out.println("O(n log n): " + maxEnvelopes(env1)); // 3
    System.out.println("O(n^2): " + maxEnvelopesDP(env1)); // 3

    int[][] env2 = {{1, 1}, {1, 1}, {1, 1}};
    System.out.println("O(n log n): " + maxEnvelopes(env2)); // 1
    System.out.println("O(n^2): " + maxEnvelopesDP(env2)); // 1
}
```

Interval DP

312. Burst Balloons

```
package leetcode.dp.interval;

import java.util.Arrays;

/**
 * 312. Burst Balloons
 * https://leetcode.com/problems/burst-balloons/
 *
 * You are given n balloons, indexed from 0 to n - 1. Each balloon is painted
 * with a number on it represented by an array nums. You are asked to burst
 * all the balloons.
 *
 * If you burst the i-th balloon, you will get nums[i-1] * nums[i] * nums[i+1] coins.
 * If i-1 or i+1 goes out of bounds, treat it as if there is a balloon with a 1 painted on it.
 *
 * Return the maximum coins you can collect by bursting the balloons wisely.
 *
 * Example 1: Input: nums = [3,1,5,8] -> Output: 167
 * Explanation: nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
 *             coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167
 *
 * Constraints:
 *   n == nums.length
 *   1 <= n <= 300
 *   0 <= nums[i] <= 100
 *
 * ---
 * Approach: Interval DP (top-down memoization)
 *
 * Key Insight: Instead of thinking about which balloon to burst FIRST,
 * think about which balloon to burst LAST in the range [i, j].
 *
 * STATE:      dp[i][j] = max coins obtainable by bursting all balloons in [i..j]
 * BASE:       dp[i][j] = 0 when i > j (no balloons)
 * TRANSITION: For each k in [i..j], burst k LAST:
 *             dp[i][j] = max(nums[i-1]*nums[k]*nums[j+1] + dp[i][k-1] + dp[k+1][j])
 *
 * Time: O(n^3)
 * Space: O(n^2)
 */
@SuppressWarnings("all")
public class BurstBalloons {

    // Top-down: find max coins from bursting all balloons in range [i..j]
    public static int helper(int[] nums, int i, int j, int[][] dp) {
        if (i > j) return 0; // No balloons left in this range

        if (dp[i][j] != -1) return dp[i][j];

        int maxCoins = 0;

        // Try every balloon k as the LAST one to burst in range [i..j]
        for (int k = i; k <= j; k++) {
            int coins = nums[i - 1] * nums[k] * nums[j + 1] // Coins from bursting k-th balloon last
                + helper(nums, i, k - 1, dp) // coins from left subarray
                + helper(nums, k + 1, j, dp); // coins from right subarray
            maxCoins = Math.max(maxCoins, coins);
        }

        dp[i][j] = maxCoins;

        return dp[i][j];
    }
}
```

```

}

public static int recursive(int[] nums) {
    int n = nums.length;
    // Pad with 1 on both sides: [1, ...original..., 1]
    int[] newNums = new int[n + 2];

    newNums[0] = 1;
    newNums[n + 1] = 1;

    for (int i = 0; i < n; i++) {
        newNums[i + 1] = nums[i];
    }

    int[][] dp = new int[n + 2][n + 2];

    for (int i = 0; i < n + 2; i++) {
        Arrays.fill(dp[i], -1);
    }

    // Solve for the range [1..n] (original balloon indices)
    return helper(newNums, 1, n, dp);
}

public static int maxCoins(int[] nums) {
    return recursive(nums);
}

public static void main(String[] args) {
    int[] nums = {3, 1, 5, 8}; // 167
    System.out.println(maxCoins(nums));
}
}

```

1547. Minimum Cost to Cut a Stick

```

package leetcode.dp.interval;

import java.util.Arrays;

/**
 * 1547. Minimum Cost to Cut a Stick
 * https://leetcode.com/problems/minimum-cost-to-cut-a-stick/
 *
 * Given a wooden stick of length n units. The stick is labelled from 0 to n.
 * Given an integer array cuts where cuts[i] denotes a position you should
 * perform a cut at. The cost of one cut is the length of the stick to be cut.
 * Return the minimum total cost of the cuts. You can change the order of cuts.
 *
 * Example 1: Input: n = 7, cuts = [1,3,4,5] -> Output: 16
 * Explanation: Order [3,5,1,4] gives cost = 7+4+3+2 = 16
 *
 * Example 2: Input: n = 9, cuts = [5,6,1,4,2] -> Output: 22
 *
 * Constraints:
 * 2 <= n <= 10^6
 * 1 <= cuts.length <= min(n - 1, 100)
 *
 * ---
 * Approach: Interval DP (top-down memoization)
 *
 * Sort cuts and add boundaries [0, n]. Then dp[left][right] = min cost to
 * make all cuts between positions left and right in the sorted cuts array.
 *
 * STATE: dp[left][right] = min cost to perform all cuts in (left..right)
 * BASE: dp[left][right] = 0 when left+1 == right (no cuts between them)

```

```

* TRANSITION: For each cut point k between left and right:
*             dp[left][right] = min(cuts[right]-cuts[left] + dp[left][k] + dp[k][right])
*
* Time:  $O(m^3)$  where  $m = \text{cuts.length}$ 
* Space:  $O(m^2)$ 
*/
@SuppressWarnings("all")
public class MinimumCostToCutStick {

    // Top-down: find min cost to make all cuts in the segment [left..right]
    public static int helper(int left, int right, int[] cuts, int[][] dp) {
        // Base case: no cuts to make between adjacent positions
        if (left + 1 == right) return 0;

        if (dp[left][right] != -1) return dp[left][right];

        int minCost = Integer.MAX_VALUE;

        // Try every cut point k between left and right
        for (int k = left + 1; k < right; k++) {
            // Cost = length of current segment + cost of left part + cost of right part
            minCost = Math.min(minCost, cuts[right] - cuts[left]
                + helper(left, k, cuts, dp) // Left Segment
                + helper(k, right, cuts, dp)); // Right Segment
        }

        dp[left][right] = minCost;

        return dp[left][right];
    }

    public static int recursive(int n, int[] cuts) {
        // Include the boundaries (0 and n) and sort all cut positions
        int m = cuts.length;
        int[] newCuts = new int[m + 2];

        for (int i = 1; i <= m; i++) {
            newCuts[i] = cuts[i - 1];
        }

        newCuts[0] = 0;
        newCuts[m + 1] = n;
        Arrays.sort(newCuts);

        int[][] dp = new int[m + 2][m + 2];

        for (int i = 0; i < m + 2; i++) {
            Arrays.fill(dp[i], -1);
        }

        // Solve for the full range [0..m+1]
        return helper(0, m + 1, newCuts, dp);
    }

    public static int minCost(int n, int[] cuts) {
        return recursive(n, cuts);
    }

    public static void main(String[] args) {
        int n = 7;
        int[] cuts = {1, 3, 4, 5}; // Output: 16

        System.out.println(recursive(n, cuts));
    }
}

```

132. Palindrome Partitioning II

```
package leetcode.dp.interval;

import java.util.HashMap;
import java.util.Map;

/**
 * 132. Palindrome Partitioning II
 * https://leetcode.com/problems/palindrome-partitioning-ii/
 *
 * Given a string s, partition s such that every substring of the partition
 * is a palindrome. Return the minimum cuts needed for a palindrome partitioning of s.
 *
 * Example 1: Input: s = "aab" -> Output: 1
 * Explanation: The palindrome partitioning ["aa","b"] needs 1 cut.
 *
 * Example 2: Input: s = "a" -> Output: 0
 *
 * Example 3: Input: s = "ab" -> Output: 1
 *
 * Constraints:
 * 1 <= s.length <= 2000
 * s consists of lowercase English letters only.
 *
 * ---
 * Approach: DP with memoization (front partitioning)
 *
 * For each starting index, try all possible palindromic substrings and
 * recursively solve the remaining suffix. Track min cuts via memoization.
 *
 * STATE:      memo[start] = min cuts to partition s[start..end]
 * BASE:       start >= s.length() -> return -1 (no cut needed, we overcount by 1)
 * TRANSITION: For each i from start to end, if s[start..i] is palindrome:
 *             memo[start] = min(1 + solve(i+1))
 *
 * Time: O(n^2) with palindrome precomputation, O(n^3) as-is
 * Space: O(n)
 */
public class PalindromePartitioning_2 {

    // Helper: check if a string is a palindrome
    public static boolean isPalindrome(String str) {
        int left = 0, right = str.length() - 1;
        while (left < right) {
            if (str.charAt(left) != str.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }

    // Top-down: find min cuts for s[start..end]
    // Returns -1 when start >= length (accounts for the extra +1 count)
    public static int backtrack(String s, int start, Map<Integer, Integer> memo) {
        if (start >= s.length()) return -1; // entire string consumed, no cut needed

        if (memo.containsKey(start)) return memo.get(start);

        int minCut = Integer.MAX_VALUE;

        // Try every possible partition point
        for (int i = start; i < s.length(); i++) {
            // If s[start..i] is a palindrome, make a cut here and solve the rest
            if (isPalindrome(s.substring(start, i + 1))) {
                minCut = Math.min(minCut, 1 + backtrack(s, i + 1, memo));
            }
        }
    }
}
```

```

        memo.put(start, minCut);

        return minCut;
    }

    public static int minCut(String s) {
        return backtrack(s, 0, new HashMap<>());
    }

    public static void main(String[] args) {
        String s = "abcd";
        System.out.println(minCut(s));
    }
}

```

486. Predict the Winner

```

package leetcode.dp.interval;

/**
 * 486. Predict the Winner
 * https://leetcode.com/problems/predict-the-winner/
 *
 * You are given an integer array nums. Two players take turns picking numbers
 * from either end of the array. Player 1 picks first. Each player adds the
 * picked number to their score. Return true if Player 1 can win or tie.
 *
 * Example 1: Input: nums = [1,5,2] -> Output: false
 * Explanation: Player 1 picks 1, Player 2 picks 5, Player 1 picks 2.
 * Score: Player 1 = 3, Player 2 = 5. Player 1 cannot win.
 *
 * Example 2: Input: nums = [1,5,233,7] -> Output: true
 *
 * Constraints:
 * 1 <= nums.length <= 20
 * 0 <= nums[i] <= 10^7
 *
 * ---
 * Approach: Interval DP (minimax with relative score)
 *
 * dp[i][j] = the maximum score difference (current player - opponent)
 * the current player can achieve from nums[i..j].
 *
 * STATE:      dp[i][j] = max relative advantage for current player in [i..j]
 * BASE:       dp[i][i] = nums[i] (only one element to pick)
 * TRANSITION: dp[i][j] = max(nums[i] - dp[i+1][j], nums[j] - dp[i][j-1])
 *              (pick left or right, then subtract opponent's best result)
 * ANSWER:     dp[0][n-1] >= 0 means Player 1 wins or ties
 *
 * Time: O(n^2)
 * Space: O(n^2)
 */
public class PredictTheWinner {

    // Top-down: returns the max score difference the current player achieves in nums[i..j]
    public static int recursive(int[] nums, int i, int j, Integer[][] dp) {
        // Base case: only one element, current player takes it
        if (i == j) return nums[i];

        if (dp[i][j] != null) return dp[i][j];

        // Pick left: gain nums[i], then opponent plays optimally on [i+1..j]
        int pickLeft = nums[i] - recursive(nums, i + 1, j, dp);
        // Pick right: gain nums[j], then opponent plays optimally on [i..j-1]
        int pickRight = nums[j] - recursive(nums, i, j - 1, dp);
    }
}

```

```

        // Choose the action that maximizes our relative advantage
        dp[i][j] = Math.max(pickLeft, pickRight);

        return dp[i][j];
    }

    public static boolean predictTheWinner(int[] nums) {
        int n = nums.length;
        Integer[][] dp = new Integer[n][n];

        // Player 1 wins if their relative score advantage >= 0
        return recursive(nums, 0, n - 1, dp) >= 0;
    }

    public static void main(String[] args) {
        int[] nums = new int[]{1, 5, 2};
        System.out.println(predictTheWinner(nums));
    }
}

```

139. Word Break

```

package leetcode.dp.interval;

import java.util.*;

/**
 * 139. Word Break
 * https://leetcode.com/problems/word-break/
 *
 * Given a string s and a dictionary of strings wordDict, return true if s
 * can be segmented into a space-separated sequence of one or more dictionary words.
 * The same word in the dictionary may be reused multiple times.
 *
 * Example 1: Input: s = "leetcode", wordDict = ["leet","code"] -> Output: true
 * Example 2: Input: s = "applepenapple", wordDict = ["apple","pen"] -> Output: true
 * Example 3: Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"] -> Output: false
 *
 * Constraints:
 * 1 <= s.length <= 300
 * 1 <= wordDict.length <= 1000
 * 1 <= wordDict[i].length <= 20
 *
 * ---
 * Approach: DP (top-down memoization + bottom-up iterative)
 *
 * STATE: dp[i] = true if s[0..i-1] can be segmented into dictionary words
 * BASE: dp[0] = true (empty prefix is trivially valid)
 * TRANSITION: dp[i] = true if there exists j < i such that dp[j] == true AND s[j..i-1] is in dict
 * ANSWER: dp[n]
 *
 * Time: O(n^2 * k) where k = average word length for substring comparison
 * Space: O(n)
 */
public class WordBreak {

    // Top-down: can we segment s[start..end] using dictionary words?
    public static boolean helper(String s, int start, Set<String> set, Map<Integer, Boolean> memo) {
        // Base case: consumed the entire string
        if (start >= s.length())
            return true;

        if (memo.containsKey(start)) {
            return memo.get(start);
        }
    }
}

```

```

    // Try every possible end point to form a word s[start..end]
    for (int end = start; end < s.length(); end++) {
        String word = s.substring(start, end + 1);

        // If this word is in the dictionary AND the rest can be segmented
        if (set.contains(word) && helper(s, end + 1, set, memo)) {
            memo.put(start, true);
            return memo.get(start);
        }
    }

    memo.put(start, false);
    return memo.get(start);
}

public static boolean recursive(String s, List<String> wordDict) {
    Set<String> set = new HashSet<>(wordDict);
    return helper(s, 0, set, new HashMap<>());
}

/**
 * dp = {
 * 8: true, // "leetcode" is fully segmented.
 * 4: true, // "code" can be segmented.
 * 0: true // "leet code" can be segmented.
 * }
 *
 * dp[i] represents: Can the substring s[0...i-1] (the prefix of length i) be successfully segmented into valid
dictionary words
 */
/**
public static boolean iterative(String s, List<String> wordDict) {
    Set<String> set = new HashSet<>(wordDict);
    int n = s.length();
    boolean[] dp = new boolean[n + 1];

    dp[0] = true;

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            String word = s.substring(j, i);
            if (dp[j] && set.contains(word)) {
                dp[i] = true;
                break;
            }
        }
    }

    return dp[n];
}

public static boolean wordBreak(String s, List<String> wordDict) {
    return iterative(s, wordDict);
}

public static void main(String[] args) {
    String s = "leetcode";
    List<String> wordDict = List.of("leet", "code");
    System.out.println(wordBreak(s, wordDict)); // Output: true

    String s2 = "applepenapple";
    List<String> wordDict2 = List.of("apple", "pen");
    System.out.println(wordBreak(s2, wordDict2)); // Output: true

    String s3 = "catsandog";
    List<String> wordDict3 = List.of("cats", "dog", "sand", "and", "cat");
    System.out.println(wordBreak(s3, wordDict3)); // Output: false
}
}

```

String DP

115. Distinct Subsequences

```
package leetcode.dp.string;

import java.util.Arrays;

/**
 * 115. Distinct Subsequences
 * https://leetcode.com/problems/distinct-subsequences/
 *
 * Given two strings s and t, return the number of distinct subsequences of s
 * which equals t.
 *
 * A subsequence of a string is a new string formed by deleting some (or no)
 * characters without disturbing the relative positions of the remaining characters.
 *
 * Example 1: Input: s = "rabbbit", t = "rabbit" -> Output: 3
 * Example 2: Input: s = "babgbag", t = "bag" -> Output: 5
 *
 * Constraints:
 * 1 <= s.length, t.length <= 1000
 * s and t consist of English letters.
 *
 * ---
 * Approach: 2D DP (top-down memoization + bottom-up iterative)
 *
 * STATE: dp[i][j] = number of ways to form t[0..i-1] from s[0..j-1]
 * BASE: dp[0][j] = 1 for all j (empty t matches any prefix of s exactly once)
 * TRANSITION: If t[i-1] == s[j-1]: dp[i][j] = dp[i-1][j-1] + dp[i][j-1]
 *             (match this char OR skip s[j-1])
 *             Else: dp[i][j] = dp[i][j-1] (skip s[j-1])
 * ANSWER: dp[t.length][s.length]
 *
 * Time: O(m * n)
 * Space: O(m * n)
 */
public class DistinctSubsequences {

    // Top-down: count ways to match t[j..] using s[i..]
    public static int helper(String s, String t, int i, int j, int[][] dp) {
        // All of t is matched
        if (j >= t.length()) return 1;
        // s is exhausted but t is not
        if (i >= s.length()) return 0;

        if (dp[i][j] != -1) return dp[i][j];

        if (s.charAt(i) == t.charAt(j)) {
            // Two choices: use s[i] to match t[j], or skip s[i]
            dp[i][j] = helper(s, t, i + 1, j + 1, dp) + helper(s, t, i + 1, j, dp);
        } else {
            // Characters don't match, must skip s[i]
            dp[i][j] = helper(s, t, i + 1, j, dp);
        }

        return dp[i][j];
    }

    public static int recursive(String s, String t) {
        int[][] dp = new int[s.length()][t.length()];
        for (int[] m : dp) Arrays.fill(m, -1);

        return helper(s, t, 0, 0, dp);
    }
}
```

```

}

/**
 * Bottom-up iterative DP
 *
 * Example: s = "babgbag", t = "bag" -> 5
 *
 * dp[i][j] = number of ways to form t[0..i-1] from s[0..j-1]
 *
 *      ""  b a b g b a g
 *      [ 1, 1, 1, 1, 1, 1, 1 ] <- base: empty t matches any prefix once
 *      b  [ 0, 1, 1, 2, 2, 3, 3 ]
 *      a  [ 0, 0, 1, 1, 1, 1, 4 ]
 *      g  [ 0, 0, 0, 0, 1, 1, 5 ] <- answer: dp[3][7] = 5
 *
 * Trace for dp[2][6] (t="ba", s="babgba"):
 *   t[1]='a' == s[5]='a' -> dp[1][5] + dp[2][5] = 3 + 1 = 4
 *   (3 ways to match 'b' in "babgb" + 1 way already found for "ba" in "babgb")
 */
public static int iterative(String s, String t) {
    int m = t.length(); // Target
    int n = s.length(); // Source
    int[][] dp = new int[m + 1][n + 1];

    // Base case: empty t can be formed from any prefix of s in exactly 1 way
    for (int j = 0; j <= n; j++) {
        dp[0][j] = 1;
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (t.charAt(i - 1) == s.charAt(j - 1)) {
                // Match s[j-1] with t[i-1] + skip s[j-1]
                dp[i][j] = dp[i - 1][j - 1] + dp[i][j - 1];
            } else {
                // Skip s[j-1]
                dp[i][j] = dp[i][j - 1];
            }
        }
    }

    return dp[t.length()][s.length()];
}

public static int numDistinct(String s, String t) {
    return recursive(s, t);
}

public static void main(String[] args) {
    String s = "rabbbit";
    String t = "rabbit";

    System.out.println(numDistinct(s, t));
}
}

```

72. Edit Distance

```

package leetcode.dp.string;

import java.util.Arrays;

/**
 * 72. Edit Distance
 * https://leetcode.com/problems/edit-distance/

```

```

*
* Given two strings word1 and word2, return the minimum number of operations
* required to convert word1 into word2. You have three operations:
* Insert a character, Delete a character, Replace a character.
*
* Example 1: Input: word1 = "horse", word2 = "ros" -> Output: 3
* Explanation: horse -> rorse (replace h with r) -> rose (remove r) -> ros (remove e)
*
* Example 2: Input: word1 = "intention", word2 = "execution" -> Output: 5
*
* Constraints:
* 0 <= word1.length, word2.length <= 500
* word1 and word2 consist of lowercase English letters.
*
* ---
* Approach: 2D DP (top-down + bottom-up)
*
* STATE:      dp[i][j] = min operations to convert word1[0..i-1] to word2[0..j-1]
* BASE:      dp[i][0] = i (delete all), dp[0][j] = j (insert all)
* TRANSITION: If chars match: dp[i][j] = dp[i-1][j-1]
*             Else: dp[i][j] = 1 + min(dp[i][j-1], dp[i-1][j], dp[i-1][j-1])
*                    (insert,      delete,      replace)
* ANSWER:    dp[m][n]
*
* Time:  O(m * n)
* Space: O(m * n)
*/
public class EditDistance {

    public static int recursive(String word1, String word2, int i, int j, int[][] dp) {
        // Base case: If one string is empty
        if (i == 0) return j; // Insert all characters of word2
        if (j == 0) return i; // Remove all characters of word1

        // Check if the result is already computed
        if (dp[i][j] != -1) {
            return dp[i][j];
        }

        // If the characters are the same, no operation is needed
        if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
            dp[i][j] = recursive(word1, word2, i - 1, j - 1, dp);
        } else {
            // Calculate the minimum of three operations (Insert, Delete, Replace)
            int insert = recursive(word1, word2, i, j - 1, dp); // Insert character from word2
            int delete = recursive(word1, word2, i - 1, j, dp); // Delete character from word1
            int replace = recursive(word1, word2, i - 1, j - 1, dp); // When you replace, it becomes equal

            dp[i][j] = 1 + Math.min(insert, Math.min(delete, replace));
        }

        return dp[i][j];
    }

    /**
     * Bottom-up iterative DP
     *
     * Example: word1 = "horse", word2 = "ros" -> 3
     *
     * dp[i][j] = min operations to convert word1[0..i-1] to word2[0..j-1]
     *
     *      ""  r  o  s
     *      [ 0, 1, 2, 3 ] <- base: insert all of word2
     *      h  [ 1, 1, 2, 3 ] h->r: replace(0+1=1)
     *      o  [ 2, 2, 1, 2 ] o==o: diagonal(1)
     *      r  [ 3, 2, 2, 2 ] r==r: diagonal(2)
     *      s  [ 4, 3, 3, 2 ] s==s: diagonal(2)
     *      e  [ 5, 4, 4, 3 ] <- answer: dp[5][3] = 3
     *
     *      ?
     *      base: delete all of word1

```

```

*
* Trace for dp[5][3] (word1="horse", word2="ros"):
* 'e' != 's' -> 1 + min(dp[4][3], dp[5][2], dp[4][2]) = 1 + min(2, 4, 3) = 3
*                               (delete) (insert) (replace)
*/
public static int iterative(String word1, String word2) {
    int m = word1.length();
    int n = word2.length();

    // dp[i][j] = min operations to convert word1[0..i-1] to word2[0..j-1]
    int[][] dp = new int[m + 1][n + 1];

    // Base case: if word1 is empty, we need to insert all characters of word2
    for (int i = 0; i <= m; i++) {
        dp[i][0] = i;
    }

    // Base case: if word2 is empty, we need to remove all characters of word1
    for (int j = 0; j <= n; j++) {
        dp[0][j] = j;
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                // Take the minimum of Insert, Delete, and Replace operations
                dp[i][j] = 1 + Math.min(dp[i - 1][j], Math.min(dp[i][j - 1], dp[i - 1][j - 1]));
            }
        }
    }

    return dp[m][n];
}

public static int minDistance(String word1, String word2) {
    int m = word1.length();
    int n = word2.length();

    int[][] dp = new int[m + 1][n + 1];
    for (int[] row : dp) {
        Arrays.fill(row, -1);
    }

    return recursive(word1, word2, m, n, dp);
}

public static void main(String[] args) {
    String word1 = "a";
    String word2 = "x";
    System.out.println(minDistance(word1, word2));
}
}

```

97. Interleaving String

```

package leetcode.dp.string;

/**
 * 97. Interleaving String
 * https://leetcode.com/problems/interleaving-string/
 *
 * Given strings s1, s2, and s3, determine whether s3 is formed by an
 * interleaving of s1 and s2. An interleaving is defined as choosing

```

```

* characters from s1 and s2 such that their relative order is preserved.
*
* Example 1: Input: s1="aabcc", s2="dbbca", s3="aadbcbcbac" -> Output: true
* Example 2: Input: s1="aabcc", s2="dbbca", s3="aadbbaaccc" -> Output: false
* Example 3: Input: s1="", s2="", s3="" -> Output: true
*
* Constraints:
*   0 <= s1.length, s2.length <= 100
*   0 <= s3.length <= 200
*   s1, s2, and s3 consist of lowercase English letters.
*
* ---
* Approach: 2D DP (top-down + bottom-up)
*
* STATE:      dp[i][j] = true if s1[0..i-1] and s2[0..j-1] interleave to form s3[0..i+j-1]
* BASE:      dp[0][0] = true; first row/col check prefix matches
* TRANSITION: dp[i][j] = (s1[i-1]==s3[k] && dp[i-1][j]) || (s2[j-1]==s3[k] && dp[i][j-1])
*             where k = i + j - 1
* ANSWER:    dp[n][m]
*
* Time: O(n * m)
* Space: O(n * m)
*/
public class InterleavingString {

    // Top-down: check if s1[i..] and s2[j..] can interleave to form s3[k..]
    public static boolean helper(String s1, String s2, String s3, int i, int j, int k, Boolean[][] dp) {
        // Both strings fully consumed -> s3 is fully matched
        if (i == s1.length() && j == s2.length()) return true;

        if (dp[i][j] != null) return dp[i][j];

        // Try taking the next character from s1
        if (i < s1.length() && s1.charAt(i) == s3.charAt(k)) {
            dp[i][j] = helper(s1, s2, s3, i + 1, j, k + 1, dp);

            if (dp[i][j]) return true;
        }

        // Try taking the next character from s2
        if (j < s2.length() && s2.charAt(j) == s3.charAt(k)) {
            dp[i][j] = helper(s1, s2, s3, i, j + 1, k + 1, dp);

            if (dp[i][j]) return true;
        }

        // Neither character matches -> interleaving not possible from here
        return dp[i][j] = false;
    }

    public static boolean recursive(String s1, String s2, String s3) {
        if (s1.length() + s2.length() != s3.length()) return false;

        Boolean[][] dp = new Boolean[s1.length() + 1][s2.length() + 1];
        helper(s1, s2, s3, 0, 0, 0, dp);

        return helper(s1, s2, s3, 0, 0, 0, dp);
    }

    /**
     * Bottom-up iterative DP
     *
     * Example: s1 = "ab", s2 = "cd", s3 = "acbd" -> true
     * Interleaving: a(s1) c(s2) b(s1) d(s2) = "acbd"
     *
     * dp[i][j] = can s1[0..i-1] and s2[0..j-1] interleave to form s3[0..i+j-1]?
     *
     *      ""    c    d
     *      "" [ T,  F,  F ]  s2[0]='c' != s3[0]='a'
     */
}

```

```

*      a   [ T,   T,   F ]   s1[0]='a'==s3[0], s2[0]='c'==s3[1]
*      b   [ F,   T,   T ]   <- answer: dp[2][2] = T
*
* Trace for dp[2][2] (s3[3]='d'):
*   s1[1]='b' != s3[3]='d' -> skip
*   s2[1]='d' == s3[3]='d' && dp[2][1]=T -> T
*/
public static boolean iterative(String s1, String s2, String s3) {
    int n = s1.length(), m = s2.length();

    if (n + m != s3.length()) return false;

    boolean[][] dp = new boolean[n + 1][m + 1];

    dp[0][0] = true;

    // Fill the first column (considering only s1)
    for (int i = 1; i <= n; i++) {
        dp[i][0] = dp[i - 1][0] && s1.charAt(i - 1) == s3.charAt(i - 1);
    }

    // Fill the first row (considering only s2)
    for (int j = 1; j <= m; j++) {
        dp[0][j] = dp[0][j - 1] && s2.charAt(j - 1) == s3.charAt(j - 1);
    }

    // Fill the DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            // current index in s3 that we are checking
            int k = (i + j) - 1;
            // Check if the current character from s1 or s2 matches the character at index k in s3.
            // If it matches, inherit truth from previous valid states.
            // dp[i][j] checks if s1[0..i-1] and s2[0..j-1] match s3[0..i+j-1].
            dp[i][j] = (s1.charAt(i - 1) == s3.charAt(k) && dp[i - 1][j]) || (s2.charAt(j - 1) == s3.charAt(k) && dp[i][j - 1]);
        }
    }

    return dp[n][m];
}

public static boolean isInterleave(String s1, String s2, String s3) {
    return true;
}

public static void main(String[] args) {
    String s1 = "aabcc";
    String s2 = "dbbca";
    String s3 = "aadbcbcbac"; // "aa dbbc bc a c";
    System.out.println(iterative(s1, s2, s3));
}
}

```

1143. Longest Common Subsequence

```

package leetcode.dp.string;

import java.util.Arrays;

/**
 * 1143. Longest Common Subsequence
 * https://leetcode.com/problems/longest-common-subsequence/
 *
 * Given two strings text1 and text2, return the length of their longest
 * common subsequence. If there is no common subsequence, return 0.

```

```

*
* Example 1: Input: text1 = "abcde", text2 = "ace" -> Output: 3
* Explanation: The longest common subsequence is "ace".
*
* Example 2: Input: text1 = "abc", text2 = "abc" -> Output: 3
* Example 3: Input: text1 = "abc", text2 = "def" -> Output: 0
*
* Constraints:
* 1 <= text1.length, text2.length <= 1000
* text1 and text2 consist of only lowercase English characters.
*
* ---
* Approach: 2D DP (top-down + bottom-up)
*
* STATE:      dp[i][j] = LCS length of text1[0..i-1] and text2[0..j-1]
* BASE:       dp[0][j] = dp[i][0] = 0 (empty string has LCS of 0)
* TRANSITION: If chars match: dp[i][j] = dp[i-1][j-1] + 1
*             Else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
* ANSWER:     dp[m][n]
*
* Time: O(m * n)
* Space: O(m * n)
*/
public class LongestCommonSubsequence {

    // Top-down: find LCS of text1[i..] and text2[j..]
    public static int helper(char[] text1, char[] text2, int i, int j, int[][] dp) {
        // Base case: one string is exhausted
        if (i >= text1.length || j >= text2.length) return 0;

        if (dp[i][j] != -1) return dp[i][j];

        if (text1[i] == text2[j]) {
            // Characters match: include in LCS and advance both
            dp[i][j] = 1 + helper(text1, text2, i + 1, j + 1, dp);
        } else {
            // Don't match: try skipping one character from each string
            int max1 = helper(text1, text2, i + 1, j, dp);
            int max2 = helper(text1, text2, i, j + 1, dp);

            dp[i][j] = Math.max(max1, max2);
        }

        return dp[i][j];
    }

    public static int recursive(String text1, String text2) {
        int[][] dp = new int[text1.length()][text2.length()];
        for (int[] m : dp) {
            Arrays.fill(m, -1);
        }
        return helper(text1.toCharArray(), text2.toCharArray(), 0, 0, dp);
    }

    /**
     * Bottom-up iterative DP
     *
     * Example: text1 = "abcde", text2 = "ace" -> 3 (LCS = "ace")
     *
     * dp[i][j] = LCS length of text1[0..i-1] and text2[0..j-1]
     *
     *      ""  a  c  e
     * ""  [ 0, 0, 0, 0 ]  <- base: LCS with empty = 0
     * a   [ 0, 1, 1, 1 ]  a==a -> 0+1=1
     * b   [ 0, 1, 1, 1 ]  b!=a,c,e -> carry forward
     * c   [ 0, 1, 2, 2 ]  c==c -> 1+1=2
     * d   [ 0, 1, 2, 2 ]  d!=a,c,e -> carry forward
    */

```

```

*      e  [ 0, 1, 2, 3 ]   e==e -> 2+1=3  <- answer
*
* Trace for dp[5][3] (text1="abcde", text2="ace"):
*   'e' == 'e' -> dp[4][2] + 1 = 2 + 1 = 3
*/
public static int iterative(String str1, String str2) {
    int m = str1.length();
    int n = str2.length();
    int[][] dp = new int[m + 1][n + 1];

    // Base cases: dp[i][0] = 0, dp[0][j] = 0 (already zero-initialized)
    for (int i = 0; i <= m; i++) {
        dp[i][0] = 0;
    }

    for (int j = 0; j <= n; j++) {
        dp[0][j] = 0;
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (str1.charAt(i - 1) == str2.charAt(j - 1)) {
                // Characters match: extend LCS by 1
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                // Skip one character from either string
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    return dp[m][n];
}

public static int longestCommonSubsequence(String text1, String text2) {
    return recursive(text1, text2);
}

public static void main(String[] args) {
    String text1 = "mom";
    String text2 = "mom";

    System.out.println(longestCommonSubsequence(text1, text2));
}
}

```

516. Longest Palindromic Subsequence

```

package leetcode.dp.string;

import java.util.Arrays;

/**
 * 516. Longest Palindromic Subsequence
 * https://leetcode.com/problems/longest-palindromic-subsequence/
 *
 * Given a string s, find the longest palindromic subsequence's length in s.
 * A subsequence is derived by deleting some or no characters without
 * changing the relative order of the remaining characters.
 *
 * Example 1: Input: s = "bbbab" -> Output: 4  ("bbbb")
 * Example 2: Input: s = "cbbd"  -> Output: 2  ("bb")
 */

```

```

* Constraints:
*   1 <= s.length <= 1000
*   s consists only of lowercase English letters.
*
* ---
* Approach: Interval DP (top-down + bottom-up)
*
* STATE:      dp[i][j] = length of longest palindromic subsequence in s[i..j]
* BASE:       dp[i][i] = 1 (single character is a palindrome)
* TRANSITION: If s[i] == s[j]: dp[i][j] = 2 + dp[i+1][j-1]
*             Else: dp[i][j] = max(dp[i+1][j], dp[i][j-1])
* ANSWER:     dp[0][n-1]
*
* Time:  O(n^2)
* Space: O(n^2)
*/
public class LongestPalindromeSubsequence {

    // Top-down: find longest palindromic subsequence in s[i..j]
    public static int helper(String s, int i, int j, int[][] dp) {
        if (i > j) return 0;           // Invalid range
        if (i == j) return 1;          // Single character palindrome

        if (dp[i][j] != -1) return dp[i][j];

        if (s.charAt(i) == s.charAt(j)) {
            // Both ends match: include them + solve inner substring
            dp[i][j] = 2 + helper(s, i + 1, j - 1, dp);
        } else {
            // Ends don't match: try excluding one end at a time
            dp[i][j] = Math.max(helper(s, i + 1, j, dp), helper(s, i, j - 1, dp));
        }

        return dp[i][j];
    }

    public static int recursive(String s) {
        int[][] dp = new int[s.length()][s.length()];
        for (int[] row : dp) Arrays.fill(row, -1);

        return helper(s, 0, s.length() - 1, dp);
    }

    /**
     * Bottom-up: fill diagonally from single chars toward full string
     *
     * Example: s = "cbdd" -> 2 (LPS = "bb")
     *
     * dp[i][j] = length of longest palindromic subsequence in s[i..j]
     *
     *      c   b   b   d
     * c [ 1,  1,  2,  2 ]  <- dp[0][3] = 2 (answer)
     * b [ -,  1,  2,  2 ]  dp[1][2]: b==b -> 2+0=2
     * b [ -,  -,  1,  1 ]  dp[2][3]: b!=d -> max(1,1)=1
     * d [ -,  -,  -,  1 ]  <- base: single chars = 1
     *
     * '-' = unused (i > j)
     *
     * Trace for dp[0][3] (s="cbdd"):
     *   s[0]='c' != s[3]='d' -> max(dp[1][3], dp[0][2]) = max(2, 2) = 2
     */
    public static int iterative(String s) {
        int n = s.length();

        int[][] dp = new int[n][n];

        // Base case: every single character is a palindrome of length 1
        for (int i = 0; i < n; i++) dp[i][i] = 1;
    }
}

```

```

// Fill column by column; for each j, process all i < j (bottom-up)
for (int j = 0; j < n; j++) {
    for (int i = j - 1; i >= 0; i--) {
        if (s.charAt(i) == s.charAt(j)) {
            dp[i][j] = 2 + dp[i + 1][j - 1];
        } else {
            dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
        }
    }
}

return dp[0][n - 1];
}

public static int longestPalindromeSubseq(String s) {
    return recursive(s);
}

public static void main(String[] args) {
    String s = "cbbd";
    System.out.println(iterative(s));
}
}

```

5. Longest Palindromic Substring

```

package leetcode.dp.string;

/**
 * 5. Longest Palindromic Substring
 * https://leetcode.com/problems/longest-palindromic-substring/
 *
 * Given a string s, return the longest palindromic substring in s.
 *
 * Example 1: Input: s = "babad" -> Output: "bab" (or "aba")
 * Example 2: Input: s = "cbbd" -> Output: "bb"
 *
 * Constraints:
 * 1 <= s.length <= 1000
 * s consist of only digits and English letters.
 *
 * ---
 * Approach: 2D DP
 *
 * dp[j][i] = true if s[j..i] is a palindrome.
 * A substring s[j..i] is a palindrome if:
 * 1. s[j] == s[i], AND
 * 2. The inner substring is a palindrome (i-j <= 2 OR dp[j+1][i-1])
 *
 * Track the longest palindrome found during the fill.
 *
 * Time: O(n^2)
 * Space: O(n^2)
 */
public class LongestPalindromicSubstring {

    /**
     * Bottom-up iterative DP
     *
     * Example: s = "babad" -> "bab"
     *
     * dp[j][i] = true if substring s[j..i] is a palindrome
     */
}

```

```

*           b       a       b       a       d
*   b [ T,   F,   T,   F,   F ]
*   a [ -,   T,   F,   T,   F ]
*   b [ -,   -,   T,   F,   F ]
*   a [ -,   -,   -,   T,   F ]
*   d [ -,   -,   -,   -,   T ]
*
* '-' = below diagonal (j > i), not used
*
* Trace for dp[0][2] (s[0..2] = "bab"):
*   s[0]='b' == s[2]='b', i-j=2 <= 2 -> T (palindrome!)
*   length 3 > current best -> result = "bab"
*
* Trace for dp[1][3] (s[1..3] = "aba"):
*   s[1]='a' == s[3]='a', i-j=2 <= 2 -> T
*   length 3 = current best -> no update
*/
public static String iterative(String s) {
    String result = "";
    boolean[][] dp = new boolean[s.length()][s.length()];

    // Base case: every single character is a palindrome
    for (int i = 0; i < s.length(); i++) {
        dp[i][i] = true;
    }

    for (int i = 0; i < s.length(); i++) {
        for (int j = 0; j <= i; j++) {
            // s[j..i] is palindrome if ends match AND inner part is valid
            // (i-j <= 2 handles length 2 and 3 substrings directly)
            if (s.charAt(i) == s.charAt(j) && (i - j <= 2 || dp[j + 1][i - 1])) {
                dp[j][i] = true;

                // Update result if this palindrome is longer
                if (i - j + 1 > result.length()) {
                    result = s.substring(j, Math.min(i + 1, s.length()));
                }
            }
        }
    }

    return result;
}

static String longestPalindrome(String s) {
    return iterative(s);
}

public static void main(String[] args) {
    String s = "babad";
    System.out.println(longestPalindrome(s));
}
}

```

10. Regular Expression Matching

```

package leetcode.dp.string;

/**
 * 10. Regular Expression Matching
 * https://leetcode.com/problems/regular-expression-matching/
 *
 * Given an input string s and a pattern p, implement regular expression matching
 * with support for '.' and '*' where:
 *   '.' Matches any single character.

```

```

* '*' Matches zero or more of the preceding element.
*
* The matching should cover the ENTIRE input string (not partial).
*
* Example 1: s = "aa", p = "a" -> false
* Example 2: s = "aa", p = "a*" -> true (a* means zero or more 'a')
* Example 3: s = "ab", p = ".*" -> true (.* means zero or more of any char)
*
* Constraints:
* 1 <= s.length <= 20
* 1 <= p.length <= 20
* s contains only lowercase English letters.
* p contains only lowercase English letters, '.', and '*'.
* Each '*' has a previous valid character to repeat.
*
* ---
* Approach: 2D DP
*
* STATE: dp[i][j] = does s[0..i-1] match p[0..j-1]?
* BASE: dp[0][0] = true (empty string matches empty pattern)
* dp[0][j] = true if p[j-1]=='*' and dp[0][j-2] (pattern like a*b*c* can match "")
* TRANSITION:
* If p[j-1] == s[i-1] or p[j-1] == '.':
* dp[i][j] = dp[i-1][j-1] (characters match, move both pointers)
* If p[j-1] == '*':
* dp[i][j] = dp[i][j-2] (use * as zero occurrences, skip X* pair)
* OR dp[i-1][j] (if p[j-2] matches s[i-1], use * for one more)
*
* KEY DIFFERENCE from Wildcard Matching (LC 44):
* - In Wildcard, '*' matches any sequence independently
* - Here, '*' modifies the PRECEDING character (X* = zero or more X)
* - So we always look at p[j-2] when handling '*'
*
* DP Matrix trace: s = "aab", p = "c*a*b"
*
*      ""  c   *   a   *   b
* "" [ T,  F,  T,  F,  T,  F ]
* a  [ F,  F,  F,  T,  T,  F ]
* a  [ F,  F,  F,  F,  T,  F ]
* b  [ F,  F,  F,  F,  F,  T ]
*
* dp[0][2]: p[1]=='*', dp[0][0]=T -> T (c* matches "")
* dp[0][4]: p[3]=='*', dp[0][2]=T -> T (c*a* matches "")
* dp[1][3]: p[2]='a' == s[0]='a', dp[0][2]=T -> T
* dp[1][4]: p[3]=='*', dp[1][2]=F (zero a) OR (p[2]='a'==s[0]='a', dp[0][4]=T) -> T
* dp[2][4]: p[3]=='*', dp[2][2]=F OR (p[2]='a'==s[1]='a', dp[1][4]=T) -> T
* dp[3][5]: p[4]='b'==s[2]='b', dp[2][4]=T -> T [v]
*
* ANSWER: dp[m][n]
*
* Time: O(m * n) where m = s.length(), n = p.length()
* Space: O(m * n)
*/

```

```

public class RegularExpressionMatching {

    // ----- Recursive + Memoization -----
    public static boolean isMatchRecursive(String s, String p) {
        Boolean[][] memo = new Boolean[s.length() + 1][p.length() + 1];
        return solve(s, p, 0, 0, memo);
    }

    private static boolean solve(String s, String p, int i, int j, Boolean[][] memo) {
        // Base case: pattern exhausted
        if (j == p.length()) return i == s.length();

        if (memo[i][j] != null) return memo[i][j];

        // Check if current characters match
        boolean firstMatch = (i < s.length()) &&
            (s.charAt(i) == p.charAt(j) || p.charAt(j) == '.');
    }

```

```

    boolean result;
    if (j + 1 < p.length() && p.charAt(j + 1) == '*') {
        // Two choices for X*:
        // 1. Skip X* entirely (zero occurrences) -> advance pattern by 2
        // 2. If first char matches, consume one char from s, keep pattern (more occurrences)
        result = solve(s, p, i, j + 2, memo) || // zero occurrences
            (firstMatch && solve(s, p, i + 1, j, memo)); // one+ occurrences
    } else {
        // No star ahead: must match current character and advance both
        result = firstMatch && solve(s, p, i + 1, j + 1, memo);
    }

    memo[i][j] = result;
    return result;
}

// ----- Iterative (Bottom-Up) -----
public static boolean isMatch(String s, String p) {
    int m = s.length();
    int n = p.length();
    boolean[][] dp = new boolean[m + 1][n + 1];

    // Empty string matches empty pattern
    dp[0][0] = true;

    // Fill first row: patterns like a*, a*b*, a*b*c* can match ""
    for (int j = 1; j <= n; j++) {
        if (p.charAt(j - 1) == '*') {
            dp[0][j] = dp[0][j - 2]; // X* matches zero X's
        }
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            char sc = s.charAt(i - 1); // Current char in s
            char pc = p.charAt(j - 1); // Current char in p

            if (pc == '.' || pc == sc) {
                // Characters match: inherit diagonal
                dp[i][j] = dp[i - 1][j - 1];
            } else if (pc == '*') {
                // Star: always try zero occurrences first
                dp[i][j] = dp[i][j - 2]; // Zero occurrences of p[j-2]

                // If the preceding pattern char matches s[i-1], try one more occurrence
                char preceding = p.charAt(j - 2);
                if (preceding == '.' || preceding == sc) {
                    dp[i][j] = dp[i][j] || dp[i - 1][j];
                    // dp[i-1][j] means: s up to i-1 already matched with same pattern,
                    // so we can extend by consuming one more character
                }
            }
            // else: mismatch, dp[i][j] stays false
        }
    }

    return dp[m][n];
}

public static void main(String[] args) {
    System.out.println(isMatch("aa", "a")); // false
    System.out.println(isMatch("aa", "a*")); // true
    System.out.println(isMatch("ab", ".*")); // true
    System.out.println(isMatch("aab", "c*a*b")); // true
    System.out.println(isMatch("mississippi", "mis*is*p*.")); // false

    System.out.println("--- Recursive ---");
    System.out.println(isMatchRecursive("aab", "c*a*b")); // true
    System.out.println(isMatchRecursive("ab", ".*")); // true
}

```

```
}
```

44. Wildcard Matching

```
package leetcode.dp.string;

import java.util.Arrays;

/**
 * 44. Wildcard Matching
 * https://leetcode.com/problems/wildcard-matching/
 *
 * Given an input string s and a pattern p, implement wildcard pattern matching
 * with support for '?' and '*' where:
 * '?' matches any single character.
 * '*' matches any sequence of characters (including the empty sequence).
 * The matching should cover the entire input string.
 *
 * Example 1: Input: s = "aa", p = "a" -> Output: false
 * Example 2: Input: s = "aa", p = "*" -> Output: true
 * Example 3: Input: s = "cb", p = "?a" -> Output: false
 *
 * Constraints:
 * 0 <= s.length, p.length <= 2000
 * s contains only lowercase English letters.
 * p contains only lowercase English letters, '?' or '*'.
 *
 * ---
 * Approach: 2D DP (top-down + bottom-up)
 *
 * STATE: dp[i][j] = true if s[0..i-1] matches p[0..j-1]
 * BASE: dp[0][0] = true; dp[0][j] = true if p[0..j-1] are all '*'
 * TRANSITION: If p[j-1] == s[i-1] or '?': dp[i][j] = dp[i-1][j-1]
 * If p[j-1] == '*': dp[i][j] = dp[i-1][j] || dp[i][j-1]
 * (match 1+ chars OR match empty)
 *
 * Time: O(m * n)
 * Space: O(m * n)
 */
public class WildcardMatching {

    private static boolean helper(String s, String p, int i, int j, int[][] dp) {
        // If the pattern is exhausted, check if the string is also exhausted
        if (j == p.length()) {
            return i == s.length();
        }

        // If the string is exhausted but the pattern is not
        if (i == s.length()) {
            // All remaining characters in the pattern must be '*'
            for (int k = j; k < p.length(); k++) {
                if (p.charAt(k) != '*') {
                    return false;
                }
            }
            return true;
        }

        // Check if the result is already computed
        if (dp[i][j] != -1) {
            return dp[i][j] == 1;
        }

        // If the characters match or the pattern has '?', move both pointers
        if (p.charAt(j) == s.charAt(i) || p.charAt(j) == '?') {
            dp[i][j] = helper(s, p, i + 1, j + 1, dp) ? 1 : 0;
        }
    }
}
```

```

// If the pattern has a '*' character
else if (p.charAt(j) == '*') {
    // Try two cases:
    // 1. '*' matches zero characters (skip the '*' in a pattern).
    // 2. '*' matches one or more characters (move to the next character in s).
    dp[i][j] = (helper(s, p, i + 1, j, dp) || helper(s, p, i, j + 1, dp)) ? 1 : 0;
} else {
    dp[i][j] = 0; // Characters do not match and no special wildcard
}

return dp[i][j] == 1;
}

public static boolean recursive(String s, String p) {
    int m = s.length();
    int n = p.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int[] row : dp) {
        Arrays.fill(row, -1);
    }

    return helper(s, p, 0, 0, dp);
}

/**
 * Bottom-up iterative DP
 *
 * Example: s = "abc", p = "a*c" -> true
 *
 * dp[i][j] = true if s[0..i-1] matches p[0..j-1]
 *
 *      ""    a    *    c
 *      "" [ T,   F,   F,   F ] <- base: empty s vs pattern
 *      a  [ F,   T,   T,   F ] 'a'=='a' -> T; '*' -> dp[0][2]||dp[1][1] = T
 *      b  [ F,   F,   T,   F ] '*' -> dp[1][2]||dp[2][1] = T (absorbs 'b')
 *      c  [ F,   F,   T,   T ] <- answer: dp[3][3] = T
 *
 * Trace for dp[3][3] (s="abc", p="a*c"):
 *   p[2]='c' == s[2]='c' -> dp[2][2] = T
 *
 * Trace for dp[2][2] (s="ab", p="a*"):
 *   p[1]='*' -> dp[1][2] || dp[2][1] = T || F = T
 *   ('*' matches "b" by extending: dp[1][2] means '*' already matched, absorb one more char)
 */
public static boolean iterative(String s, String p) {
    int m = s.length();
    int n = p.length();
    boolean[][] dp = new boolean[m + 1][n + 1];

    // Empty pattern matches empty string
    dp[0][0] = true;

    // Fill in the first row (when string is empty but the pattern is not)
    for (int j = 1; j <= n; j++) {
        if (p.charAt(j - 1) == '*') {
            dp[0][j] = dp[0][j - 1]; // '*' can match an empty sequence
        }
    }

    // Fill the DP table
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (p.charAt(j - 1) == s.charAt(i - 1) || p.charAt(j - 1) == '?') {
                dp[i][j] = dp[i - 1][j - 1]; // Match the current characters
            } else if (p.charAt(j - 1) == '*') {
                dp[i][j] = dp[i - 1][j] || dp[i][j - 1]; // '*' matches zero or more characters
            }
        }
    }
}

```

```
        }  
    }  
  
    return dp[m][n];  
}  
  
public static boolean isMatch(String s, String p) {  
    return recursive(s, p);  
}  
  
public static void main(String[] args) {  
    System.out.println(iterative("abc", "a*b*"));  
}  
}
```

Stock Buy & Sell

121. Best Time to Buy and Sell Stock

```
package leetcode.dp.stock;

/**
 * 121. Best Time to Buy and Sell Stock
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock/
 *
 * You are given an array prices where prices[i] is the price of a given stock
 * on the i-th day. You want to maximize your profit by choosing a single day
 * to buy and a single day in the future to sell. Return the maximum profit.
 * If you cannot achieve any profit, return 0.
 *
 * Example 1: Input: prices = [7,1,5,3,6,4] -> Output: 5
 * Explanation: Buy on day 2 (price=1) and sell on day 5 (price=6), profit = 5.
 *
 * Example 2: Input: prices = [7,6,4,3,1] -> Output: 0
 *
 * Constraints:
 * 1 <= prices.length <= 10^5
 * 0 <= prices[i] <= 10^4
 *
 * ---
 * Approach: Greedy / One-pass
 *
 * Track the minimum price seen so far. At each day, the max profit is
 * current price - minimum price seen. Keep a running maximum.
 *
 * Time: O(n)
 * Space: O(1)
 */
public class BestTimeBuySellStock {

    public static int maxProfit(int[] prices) {
        int minPrice = prices[0]; // Minimum buy price seen so far
        int maxProfit = 0; // Best profit achievable

        for (int price : prices) {
            // Update the cheapest buy price
            minPrice = Math.min(minPrice, price);

            // Check if selling today gives a better profit
            if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }

        return maxProfit;
    }

    public static void main(String[] args) {
        int[] prices = { 7, 1, 5, 3, 6, 4 };
        System.out.println(maxProfit(prices));
    }
}
```

309. Best Time to Buy and Sell Stock with Cooldown

```
package leetcode.dp.stock;
```

```

/**
 * 309. Best Time to Buy and Sell Stock with Cooldown
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-with-cooldown/
 *
 * You are given an array prices where prices[i] is the price on the i-th day.
 * Find the maximum profit with as many transactions as you like, but after
 * selling you must wait one day before buying again (cooldown of 1 day).
 *
 * Example 1: Input: prices = [1,2,3,0,2] -> Output: 3
 * Explanation: [buy, sell, cooldown, buy, sell]
 *
 * Example 2: Input: prices = [1] -> Output: 0
 *
 * Constraints:
 * 1 <= prices.length <= 5000
 * 0 <= prices[i] <= 1000
 *
 * ---
 * Approach: State Machine DP with 3 states
 *
 * On each day, you are in one of three states:
 * hold:    holding a stock (bought previously or today)
 * sold:    just sold a stock today
 * cooldown: not holding, did NOT sell today (either idle or post-cooldown)
 *
 * Transitions:
 * hold     = max(hold, cooldown - price) // keep holding OR buy today from cooldown
 * sold     = hold + price                // sell the stock we hold
 * cooldown = max(cooldown, sold)        // stay idle OR transition from yesterday's sold
 *
 * Time: O(n)
 * Space: O(1)
 */
public class BestTimeBuySellStockCooldown {

    public static int maxProfit(int[] prices) {
        if (prices == null || prices.length == 0) {
            return 0;
        }

        int n = prices.length;

        int hold = -prices[0]; // Bought on day 0
        int sold = 0;          // Haven't sold anything yet
        int cooldown = 0;      // Haven't done anything yet

        for (int i = 1; i < n; i++) {
            int prevHold = hold;

            // hold: keep holding OR buy today (must come from cooldown state)
            hold = Math.max(hold, cooldown - prices[i]);
            // cooldown: stay in cooldown OR move from yesterday's sold
            cooldown = Math.max(cooldown, sold);
            // sold: sell the stock we're holding today
            sold = prevHold + prices[i];
        }

        // Answer is max of sold or cooldown (never profitable to end holding)
        return Math.max(sold, cooldown);
    }

    public static void main(String[] args) {
        // Test case 1
        int[] prices1 = { 1, 2, 3, 0, 2 };
        System.out.println(maxProfit(prices1)); // Output: 3

        // Test case 2
        int[] prices2 = { 1 };
        System.out.println(maxProfit(prices2)); // Output: 0
    }
}

```

```
}

```

714. Best Time to Buy and Sell Stock with Transaction Fee

```
package leetcode.dp.stock;

/**
 * 714. Best Time to Buy and Sell Stock with Transaction Fee
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-with-transaction-fee/
 *
 * You are given an array prices where prices[i] is the price of a given stock
 * on the i-th day, and an integer fee representing a transaction fee.
 *
 * Find the maximum profit you can achieve. You may complete as many transactions
 * as you like, but you need to pay the transaction fee for each transaction.
 * You may not engage in multiple transactions simultaneously (you must sell the
 * stock before you buy again).
 *
 * The transaction fee is only charged once for each stock purchase and sale.
 *
 * Example 1:
 * Input: prices = [1, 3, 2, 8, 4, 9], fee = 2
 * Output: 8
 * Explanation: Buy at 1, sell at 8 (profit 8-1-2=5), buy at 4, sell at 9 (profit 9-4-2=3)
 * Total = 5 + 3 = 8
 *
 * Example 2:
 * Input: prices = [1, 3, 7, 5, 10, 3], fee = 3
 * Output: 6
 *
 * Constraints:
 * 1 <= prices.length <= 5 * 10^4
 * 1 <= prices[i] < 5 * 10^4
 * 0 <= fee < 5 * 10^4
 *
 * ---
 * Approach: State Machine DP (same as Buy/Sell Stock II, but subtract fee on sell)
 *
 * Two states at each day:
 * cash = max profit when NOT holding stock
 * hold = max profit when HOLDING stock
 *
 * Transitions:
 * cash = max(cash, hold + prices[i] - fee) // rest or sell (pay fee)
 * hold = max(hold, cash - prices[i]) // rest or buy
 *
 * Base: cash = 0, hold = -prices[0] (buy on day 0)
 *
 * This is identical to BestTimeBuySellStock_2 with unlimited transactions,
 * except we subtract 'fee' each time we sell.
 *
 * Trace: prices = [1, 3, 2, 8, 4, 9], fee = 2
 *
 * Day 0 (price=1): cash=0, hold=-1
 * Day 1 (price=3): cash=max(0, -1+3-2)=0, hold=max(-1, 0-3)=-1
 * Day 2 (price=2): cash=max(0, -1+2-2)=0, hold=max(-1, 0-2)=-1
 * Day 3 (price=8): cash=max(0, -1+8-2)=5, hold=max(-1, 5-8)=-1
 * Day 4 (price=4): cash=max(5, -1+4-2)=5, hold=max(-1, 5-4)=1
 * Day 5 (price=9): cash=max(5, 1+9-2)=8, hold=max(1, 8-9)=1
 * Answer: cash = 8 [v]
 *
 * Time: O(n)
 * Space: O(1)
 */
public class BestTimeBuySellStockWithTransactionFee {

    // ----- Optimal O(1) Space -----

```

```

public static int maxProfit(int[] prices, int fee) {
    // cash: best profit ending in "not holding" state
    // hold: best profit ending in "holding" state
    int cash = 0;
    int hold = -prices[0]; // Buy on day 0

    for (int i = 1; i < prices.length; i++) {
        // Sell: transition from hold -> cash, pay fee
        int newCash = Math.max(cash, hold + prices[i] - fee);
        // Buy: transition from cash -> hold
        int newHold = Math.max(hold, cash - prices[i]);

        cash = newCash;
        hold = newHold;
    }

    // Must end in "not holding" state for maximum profit
    return cash;
}

// ----- Recursive + Memoization -----
public static int maxProfitRecursive(int[] prices, int fee) {
    int[][] memo = new int[prices.length][2];
    for (int[] row : memo) java.util.Arrays.fill(row, -1);
    return solve(prices, fee, 0, 0, memo);
}

/**
 * @param idx    current day index
 * @param hold   0 = not holding, 1 = holding
 */
private static int solve(int[] prices, int fee, int idx, int hold, int[][] memo) {
    if (idx == prices.length) return 0;
    if (memo[idx][hold] != -1) return memo[idx][hold];

    // Option 1: Do nothing (rest)
    int rest = solve(prices, fee, idx + 1, hold, memo);

    if (hold == 1) {
        // Option 2: Sell today (pay fee)
        int sell = prices[idx] - fee + solve(prices, fee, idx + 1, 0, memo);
        memo[idx][hold] = Math.max(rest, sell);
    } else {
        // Option 2: Buy today
        int buy = -prices[idx] + solve(prices, fee, idx + 1, 1, memo);
        memo[idx][hold] = Math.max(rest, buy);
    }

    return memo[idx][hold];
}

public static void main(String[] args) {
    int[] prices1 = {1, 3, 2, 8, 4, 9};
    System.out.println("Example 1: " + maxProfit(prices1, 2)); // 8
    System.out.println("Recursive: " + maxProfitRecursive(prices1, 2)); // 8

    int[] prices2 = {1, 3, 7, 5, 10, 3};
    System.out.println("Example 2: " + maxProfit(prices2, 3)); // 6
}
}

```

122. Best Time to Buy and Sell Stock II

```

package leetcode.dp.stock;

/**
 * 122. Best Time to Buy and Sell Stock II

```

```

* https://leetcode.com/problems/best-time-to-buy-and-sell-stock-ii/
*
* You are given an integer array prices where prices[i] is the price of a
* given stock on the i-th day. On each day, you may decide to buy and/or
* sell the stock. You can hold at most one share at a time. You can buy and
* sell on the same day. Find the maximum profit you can achieve.
*
* Example 1: Input: prices = [7,1,5,3,6,4] -> Output: 7
* Explanation: Buy day 2, sell day 3 (profit 4). Buy day 4, sell day 5 (profit 3). Total = 7.
*
* Example 2: Input: prices = [1,2,3,4,5] -> Output: 4
* Example 3: Input: prices = [7,6,4,3,1] -> Output: 0
*
* Constraints:
* 1 <= prices.length <= 3 * 10^4
* 0 <= prices[i] <= 10^4
*
* ---
* Approach: Greedy
*
* Collect every upward price movement. Whenever tomorrow's price is higher
* than today's, add the difference as profit. This is equivalent to buying
* and selling on every consecutive profitable pair.
*
* Time: O(n)
* Space: O(1)
*/
public class BestTimeBuySellStock_2 {

    public static int maxProfit(int[] prices) {
        int profit = 0;

        // Accumulate profit from every upward price movement
        for (int i = 1; i < prices.length; i++) {
            if (prices[i] > prices[i - 1]) {
                profit += prices[i] - prices[i - 1];
            }
        }

        return profit;
    }

    public static void main(String[] args) {
        // Test case 1
        int[] prices1 = { 7, 1, 5, 3, 6, 4 };
        System.out.println(maxProfit(prices1)); // Output: 7

        // Test case 2
        int[] prices2 = { 1, 2, 3, 4, 5 };
        System.out.println(maxProfit(prices2)); // Output: 4

        // Test case 3
        int[] prices3 = { 7, 6, 4, 3, 1 };
        System.out.println(maxProfit(prices3)); // Output: 0
    }
}

```

123. Best Time to Buy and Sell Stock III

```

package leetcode.dp.stock;

/**
 * 123. Best Time to Buy and Sell Stock III
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-iii/
 *
 * You are given an array prices where prices[i] is the price of a given stock
 * on the i-th day. Find the maximum profit you can achieve. You may complete

```

```

* at most two transactions. You may not engage in multiple transactions
* simultaneously (you must sell the stock before you buy again).
*
* Example 1: Input: prices = [3,3,5,0,0,3,1,4] -> Output: 6
* Explanation: Buy day 4 (price=0), sell day 6 (price=3), profit=3.
*              Buy day 7 (price=1), sell day 8 (price=4), profit=3. Total=6.
*
* Example 2: Input: prices = [1,2,3,4,5] -> Output: 4
* Example 3: Input: prices = [7,6,4,3,1] -> Output: 0
*
* Constraints:
* 1 <= prices.length <= 10^5
* 0 <= prices[i] <= 10^5
*
* ---
* Approach: State Machine / DP in O(1) space
*
* Track 4 variables across one pass:
* minPrice1: cheapest buy for transaction 1
* profit1:   max profit after selling transaction 1
* minPrice2: effective cheapest buy for transaction 2 (price - profit1)
* profit2:   max profit after selling transaction 2
*
* The trick: minPrice2 = min(price - profit1) accounts for the profit
* already earned from the first transaction, so profit2 captures the
* combined profit of both transactions.
*
* Time: O(n)
* Space: O(1)
*/
public class BestTimeBuySellStock_3 {

    public static int maxProfit(int[] prices) {
        if (prices == null || prices.length == 0) {
            return 0;
        }

        int minPrice1 = Integer.MAX_VALUE; // Min buy price for 1st transaction
        int minPrice2 = Integer.MAX_VALUE; // Effective min buy price for 2nd transaction
        int profit1 = 0; // Max profit from 1st transaction
        int profit2 = 0; // Max profit from both transactions

        for (int price : prices) {
            // Transaction 1: find best single buy-sell
            minPrice1 = Math.min(minPrice1, price);
            profit1 = Math.max(profit1, price - minPrice1);

            // Transaction 2: effective buy price = price - profit1
            // This "reinvests" profit1 so profit2 captures total of both
            minPrice2 = Math.min(minPrice2, price - profit1);
            profit2 = Math.max(profit2, price - minPrice2);
        }

        return profit2;
    }

    public static void main(String[] args) {
        // Test case 1
        int[] prices1 = { 3, 3, 5, 0, 0, 3, 1, 4 };
        System.out.println(maxProfit(prices1)); // Output: 6

        // Test case 2
        int[] prices2 = { 1, 2, 3, 4, 5 };
        System.out.println(maxProfit(prices2)); // Output: 4

        // Test case 3
        int[] prices3 = { 7, 6, 4, 3, 1 };
        System.out.println(maxProfit(prices3)); // Output: 0
    }
}

```

188. Best Time to Buy and Sell Stock IV

```

package leetcode.dp.stock;

import java.util.Arrays;

/**
 * 188. Best Time to Buy and Sell Stock IV
 * https://leetcode.com/problems/best-time-to-buy-and-sell-stock-iv/
 *
 * You are given an integer array prices where prices[i] is the price of a
 * given stock on the i-th day, and an integer k.
 * Find the maximum profit you can achieve with at most k transactions.
 * You may not hold more than one share at a time.
 *
 * Example 1: Input: k = 2, prices = [2,4,1] -> Output: 2
 * Explanation: Buy day 1 (price=2), sell day 2 (price=4), profit = 2.
 *
 * Example 2: Input: k = 2, prices = [3,2,6,5,0,3] -> Output: 7
 * Explanation: Buy day 2, sell day 3 (profit 4). Buy day 5, sell day 6 (profit 3). Total=7.
 *
 * Constraints:
 * 1 <= k <= 100
 * 1 <= prices.length <= 1000
 * 0 <= prices[i] <= 1000
 *
 * ---
 * Approach: DP with buy[] and sell[] arrays
 *
 * Generalization of Stock III. Maintain k buy/sell states:
 * buy[i] = max profit when holding stock after i-th buy
 * sell[i] = max profit when NOT holding stock after i-th sell
 *
 * For each price, update all k transactions:
 * buy[i] = max(buy[i], sell[i-1] - price) // buy using profit from previous sell
 * sell[i] = max(sell[i], buy[i] + price) // sell the stock we hold
 *
 * Time: O(n * k)
 * Space: O(k)
 */
public class BestTimeBuySellStock_4 {

    public static int maxProfit(int k, int[] prices) {
        int[] buy = new int[k + 1]; // buy[i] = max profit in "holding" state for i-th transaction
        int[] sell = new int[k + 1]; // sell[i] = max profit in "sold" state for i-th transaction

        // Initialize buy states to -infinity (haven't bought yet)
        Arrays.fill(buy, Integer.MIN_VALUE);

        for (int price : prices) {
            for (int i = 1; i <= k; i++) {
                // Either keep holding, or buy today using previous sell's profit
                buy[i] = Math.max(buy[i], sell[i - 1] - price);
                // Either keep waiting, or sell today
                sell[i] = Math.max(sell[i], buy[i] + price);
            }
        }
        return sell[k];
    }

    public static void main(String[] args) {
        // Test case 1
        int[] prices1 = { 2, 4, 1 };
        System.out.println(maxProfit(2, prices1)); // Output: 2

        // Test case 2
        int[] prices2 = { 3, 2, 6, 5, 0, 3 };
        System.out.println(maxProfit(2, prices2)); // Output: 7
    }
}

```

```
}
```

Digit DP

357. Count Numbers with Unique Digits

```
package leetcode.dp.digit;

/**
 * 357. Count Numbers with Unique Digits
 * https://leetcode.com/problems/count-numbers-with-unique-digits/
 *
 * Given an integer n, return the count of all numbers with unique digits, x,
 * where 0 ≤ x < 10^n.
 *
 * Example 1: Input: n = 2 -> Output: 91
 * Explanation: All numbers from 0 to 99 except [11,22,33,44,55,66,77,88,99]
 * have unique digits. 100 - 9 = 91.
 *
 * Example 2: Input: n = 0 -> Output: 1 (only 0)
 * Example 3: Input: n = 1 -> Output: 10 (0 through 9)
 *
 * Constraints:
 * 0 ≤ n ≤ 8
 *
 * ---
 * Approach: Combinatorics / Math DP
 *
 * Count k-digit numbers with unique digits for each k from 1 to n.
 * For a k-digit number:
 * - First digit: 9 choices (1-9, can't be 0)
 * - Second digit: 9 choices (0-9 minus the first)
 * - Third digit: 8 choices, and so on.
 *
 * uniqueK = 9 * 9 * 8 * 7 * ... * (11 - k)
 * answer = 1 + sum(uniqueK) for k = 1..n
 *
 * Time: O(n)
 * Space: O(1)
 */
public class CountNumbersWithUniqueDigits {

    public static int countNumbersWithUniqueDigits(int n) {
        if (n == 0) return 1;

        int total = 10; // For n=1, we have 0..9 = 10 numbers
        int uniqueK = 9; // Count of 1-digit unique-digit numbers (1-9)

        for (int k = 2; k ≤ n && k ≤ 10; k++) {
            // For k-th digit, we have (11 - k) remaining choices
            // k=2: 9 choices (0-9 minus 1 used), k=3: 8 choices, etc.
            uniqueK *= (11 - k);
            total += uniqueK;
        }

        return total;
    }

    /**
     * Alternative: Digit DP with bitmask
     *
     * This approach generalizes better to harder digit DP problems.
     * dp(pos, mask, started) = count of valid numbers from position pos onward.
     *
     * - pos: current digit position (0 to n-1)
     * - mask: bitmask of digits already used (10 bits for 0-9)
     * - started: whether we've placed a non-zero digit yet (handles leading zeros)
     */
}
```

```

public static int digitDP(int n) {
    if (n == 0) return 1;
    // memo[pos][mask][started] -- mask has 1024 states, started has 2
    Integer[][][] memo = new Integer[n][1 << 10][2];
    return 1 + solve(0, 0, false, n, memo); // +1 for zero itself
}

private static int solve(int pos, int mask, boolean started, int n, Integer[][][] memo) {
    // All positions filled: we've formed one valid number
    if (pos == n) return started ? 1 : 0;

    int s = started ? 1 : 0;
    if (memo[pos][mask][s] != null) return memo[pos][mask][s];

    int count = 0;

    for (int d = 0; d <= 9; d++) {
        // Skip if digit already used
        if ((mask & (1 << d)) != 0) continue;

        if (!started && d == 0) {
            // Leading zero: don't mark it in the mask, keep started=false
            count += solve(pos + 1, mask, false, n, memo);
        } else {
            // Place digit d: mark it in the mask, started=true
            count += solve(pos + 1, mask | (1 << d), true, n, memo);
        }
    }

    memo[pos][mask][s] = count;
    return count;
}

public static void main(String[] args) {
    System.out.println("n=0: " + countNumbersWithUniqueDigits(0)); // 1
    System.out.println("n=1: " + countNumbersWithUniqueDigits(1)); // 10
    System.out.println("n=2: " + countNumbersWithUniqueDigits(2)); // 91
    System.out.println("n=3: " + countNumbersWithUniqueDigits(3)); // 739

    System.out.println("--- Digit DP ---");
    System.out.println("n=0: " + digitDP(0)); // 1
    System.out.println("n=2: " + digitDP(2)); // 91
    System.out.println("n=3: " + digitDP(3)); // 739
}
}

```

233. Number of Digit One

```

package leetcode.dp.digit;

/**
 * 233. Number of Digit One
 * https://leetcode.com/problems/number-of-digit-one/
 *
 * Given an integer n, count the total number of digit 1 appearing in all
 * non-negative integers less than or equal to n.
 *
 * Example 1: Input: n = 13 -> Output: 6
 * Explanation: Digit 1 appears in: 1, 10, 11 (twice), 12, 13 -> total 6.
 *
 * Example 2: Input: n = 0 -> Output: 0
 *
 * Constraints:
 * 0 <= n <= 10^9
 *
 * ---
 * Approach: Digit DP
 */

```

```

*
* Process the number digit by digit (as a string). At each position, decide
* what digit to place and track state:
*
* dp(pos, count, tight) = total count of digit '1' across all valid numbers
* - pos:    current digit position being filled
* - count:  how many 1s placed so far in this number
* - tight:  whether we're still bounded by n's prefix (can't exceed n)
*
* If tight=true, digit can go from 0 to digits[pos].
* If tight=false, digit can be 0 to 9 freely.
*
* We count 1s contributed by ALL numbers from 0 to n collectively.
*
* Time:  O(len * len * 2) where len = number of digits in n (~10)
* Space: O(len * len * 2)
*/
public class NumberOfDigitOne {

    /**
     * Digit DP Solution
     */
    public static int countDigitOne(int n) {
        if (n <= 0) return 0;

        String num = String.valueOf(n);
        int len = num.length();
        // memo[pos][count][tight]
        // count can be at most len (all digits are 1), tight is 0 or 1
        Integer[][][] memo = new Integer[len][len + 1][2];

        return solve(0, 0, true, num, memo);
    }

    /**
     * @param pos    current digit position (0-indexed from left)
     * @param count  number of 1s placed so far
     * @param tight  true if previous digits exactly matched n's prefix
     * @param num    string representation of n
     * @param memo   memoization table
     * @return total count of digit 1 in all numbers formed from pos onward
     */
    private static int solve(int pos, int count, boolean tight, String num, Integer[][][] memo) {
        // All positions filled: return the count of 1s in this number
        if (pos == num.length()) return count;

        int t = tight ? 1 : 0;
        if (memo[pos][count][t] != null) return memo[pos][count][t];

        // Upper limit for this digit position
        int limit = tight ? (num.charAt(pos) - '0') : 9;

        int result = 0;
        for (int d = 0; d <= limit; d++) {
            // If we place digit 1, count increases by 1
            int newCount = count + (d == 1 ? 1 : 0);
            // tight remains true only if we're still matching n's digits
            boolean newTight = tight && (d == limit);
            result += solve(pos + 1, newCount, newTight, num, memo);
        }

        memo[pos][count][t] = result;
        return result;
    }

    /**
     * Mathematical approach (O(log n) per digit position)
     *
     * For each position, count how many times digit 1 appears there
     * across all numbers from 0 to n.
     */

```

```

*
* For position i (from right, 0-indexed), let:
*   factor = 10^i
*   higher = n / (factor * 10)
*   curr   = (n / factor) % 10
*   lower  = n % factor
*
* Count of 1s at position i:
*   if curr == 0: higher * factor
*   if curr == 1: higher * factor + lower + 1
*   if curr >= 2: (higher + 1) * factor
*/
public static int countDigitOneMath(int n) {
    int count = 0;
    // factor represents 10^i (the place value: 1, 10, 100, ...)
    for (long factor = 1; factor <= n; factor *= 10) {
        long higher = n / (factor * 10); // Digits above current position
        long curr = (n / factor) % 10;   // Current digit
        long lower = n % factor;         // Digits below current position

        if (curr == 0) {
            count += higher * factor;
        } else if (curr == 1) {
            count += higher * factor + lower + 1;
        } else {
            count += (higher + 1) * factor;
        }
    }
    return count;
}

public static void main(String[] args) {
    System.out.println("n=13 (DP): " + countDigitOne(13)); // 6
    System.out.println("n=13 (Math): " + countDigitOneMath(13)); // 6
    System.out.println("n=100 (DP): " + countDigitOne(100)); // 21
    System.out.println("n=100 (Math): " + countDigitOneMath(100)); // 21
}
}

```

902. Numbers At Most N Given Digit Set

```

package leetcode.dp.digit;

/**
 * 902. Numbers At Most N Given Digit Set
 * https://leetcode.com/problems/numbers-at-most-n-given-digit-set/
 *
 * Given an array of digits (sorted strings like ["1","3","5","7"]), return
 * the number of positive integers that can be generated that are less than
 * or equal to n. Each digit in digits[] can be used any number of times.
 *
 * Example 1: Input: digits = ["1","3","5","7"], n = 100 -> Output: 20
 * Explanation: 1-digit: 1,3,5,7 (4). 2-digit: 11,13,15,17,31,...,77 (16). Total = 20.
 *
 * Example 2: Input: digits = ["1","4","9"], n = 1000000000 -> Output: 29523
 *
 * Example 3: Input: digits = ["7"], n = 8 -> Output: 1
 *
 * Constraints:
 *   1 <= digits.length <= 9
 *   digits[i].length == 1
 *   digits[i] is a digit from '1' to '9'
 *   All values in digits are unique and sorted
 *   1 <= n <= 10^9
 *
 * ---
 * Approach: Digit DP

```

```

*
* Two parts:
* 1. Count all numbers with FEWER digits than n (they're all valid).
*    For k digits: digits.length^k choices.
*
* 2. Count numbers with the SAME number of digits as n that are <= n.
*    Use digit DP: at each position, either place a digit less than n's digit
*    (then remaining positions are free) or match n's digit exactly (stay tight).
*
* Time: O(log n * |digits|)
* Space: O(log n)
*/
public class NumbersAtMostNGivenDigitSet {

    public static int atMostNGivenDigitSet(String[] digits, int n) {
        String num = String.valueOf(n);
        int len = num.length();
        int d = digits.length;

        int count = 0;

        // Part 1: Count all numbers with fewer digits than n.
        // For k-digit numbers, each position has 'd' choices: d^k total.
        for (int k = 1; k < len; k++) {
            count += pow(d, k);
        }

        // Part 2: Count numbers with exactly 'len' digits that are <= n.
        // Digit DP: at each position, try all digits.
        count += digitDP(0, true, num, digits);

        return count;
    }

    /**
     * Digit DP: count valid numbers from position pos onward.
     *
     * @param pos    current position in the number
     * @param tight  are we still bounded by n's prefix?
     * @param num    string form of n
     * @param digits available digits
     */
    private static int digitDP(int pos, boolean tight, String num, String[] digits) {
        if (pos == num.length()) {
            return 1; // We've formed a valid number
        }

        int limit = num.charAt(pos) - '0';
        int d = digits.length;
        int count = 0;

        for (String digit : digits) {
            int dig = digit.charAt(0) - '0';

            if (tight) {
                if (dig < limit) {
                    // Placed a digit smaller than n's: remaining positions are free
                    count += pow(d, num.length() - pos - 1);
                } else if (dig == limit) {
                    // Matched n's digit: stay tight for next position
                    count += digitDP(pos + 1, true, num, digits);
                }
                // dig > limit: skip (would exceed n)
            } else {
                // Not tight: any digit is fine, all remaining positions are free
                count += pow(d, num.length() - pos - 1);
            }
        }

        return count;
    }
}

```

```
}

private static int pow(int base, int exp) {
    int result = 1;
    for (int i = 0; i < exp; i++) {
        result *= base;
    }
    return result;
}

public static void main(String[] args) {
    String[] digits1 = {"1", "3", "5", "7"};
    System.out.println("digits=[1,3,5,7], n=100: " + atMostNGivenDigitSet(digits1, 100)); // 20

    String[] digits2 = {"1", "4", "9"};
    System.out.println("digits=[1,4,9], n=1000000000: " + atMostNGivenDigitSet(digits2, 1000000000)); // 29523

    String[] digits3 = {"7"};
    System.out.println("digits=[7], n=8: " + atMostNGivenDigitSet(digits3, 8)); // 1
}
}
```

Bitmask DP

473. Matchsticks to Square

```
package leetcode.dp.bitmask;

import java.util.Arrays;

/**
 * 473. Matchsticks to Square
 * https://leetcode.com/problems/matchsticks-to-square/
 *
 * You are given an integer array matchsticks where matchsticks[i] is the length
 * of the i-th matchstick. You want to use all the matchsticks to make one square.
 * You should not break any stick, but you can link them up, and each matchstick
 * must be used exactly once.
 *
 * Return true if you can make this square and false otherwise.
 *
 * Example 1: Input: matchsticks = [1,1,2,2,2] -> Output: true
 * Explanation: You can form a square with side length 2,
 * one side of the square is formed by two sticks of length 1.
 *
 * Example 2: Input: matchsticks = [3,3,3,3,4] -> Output: false
 * Explanation: You cannot find a way to form a square with all the matchsticks.
 *
 * Constraints:
 * 1 <= matchsticks.length <= 15
 * 1 <= matchsticks[i] <= 10^8
 *
 * ---
 * Approach: Bitmask DP
 *
 * Since n <= 15, we can represent every subset of matchsticks as a bitmask (2^15 = 32768 states).
 *
 * Key Idea:
 * We greedily fill one side of the square at a time. dp[mask] tracks the
 * "remainder" length on the current (incomplete) side after placing all sticks
 * in the subset represented by mask.
 *
 * When dp[mask] reaches exactly `target` (side length), we take modulo to reset
 * it to 0, meaning that side is complete and we start filling the next side.
 *
 * If dp[(1<n) - 1] == 0, all sticks are used and all four sides are exactly filled.
 *
 * Time: O(2^n * n) -- for each of the 2^n masks, we try n sticks.
 * Space: O(2^n) -- for the dp array.
 */
public class MatchsticksToSquare {

    /**
     * STATE: dp[mask]
     * For a subset of sticks (mask), dp[mask] stores how many units are
     * currently contributing to the "incomplete" side.
     */
    public boolean makeSquare(int[] matchsticks) {
        int n = matchsticks.length;
        if (n < 4)
            return false;

        long totalSum = 0;
        for (int s : matchsticks)
            totalSum += s;

        // If the total length isn't divisible by 4, we can't form a square
        if (totalSum % 4 != 0)
```

```

        return false;

    int target = (int) (totalSum / 4);
    int numStates = 1 << n;
    int[] dp = new int[numStates];

    // BASE CASE:
    // Start with an empty set of sticks.
    // Initialize all other states as unreachable (-1).
    Arrays.fill(dp, -1);
    dp[0] = 0;

    // TRANSITION:
    // For every reachable state (mask), try adding an unused matchstick (j).
    for (int mask = 0; mask < numStates; mask++) {
        if (dp[mask] == -1)
            continue;

        for (int j = 0; j < n; j++) {
            // Check if the j-th matchstick is already used in the current mask
            if ((mask & (1 << j)) == 0) {
                // Include the current matchstick in the mask
                int nextMask = mask | (1 << j);

                // If adding this stick doesn't exceed the target side length...
                if (dp[mask] + matchsticks[j] <= target) {
                    // Modulo resets side length to 0 once 'target' is reached, starting the next side.
                    // Goal state: all bits set (11...1) indicates all matchsticks have been used.
                    // Goal state: (1 << n) - 1 indicates every stick has been placed.
                    dp[nextMask] = (dp[mask] + matchsticks[j]) % target;
                }
            }
        }
    }

    return dp[numStates - 1] == 0;
}

public static void main(String[] args) {
    MatchsticksToSquare solver = new MatchsticksToSquare();

    // Test Case 1
    int[] matchsticks1 = { 1, 1, 2, 2, 2 };
    System.out.println("Test Case 1: " + Arrays.toString(matchsticks1));
    System.out.println("Can form square? " + solver.makeSquare(matchsticks1)); // Expected: true

    System.out.println("---");

    // Test Case 2
    int[] matchsticks2 = { 3, 3, 3, 3, 4 };
    System.out.println("Test Case 2: " + Arrays.toString(matchsticks2));
    System.out.println("Can form square? " + solver.makeSquare(matchsticks2)); // Expected: false
}
}

```

1125. Smallest Sufficient Team

```

package leetcode.dp.bitmask;

import java.util.*;

/**
 * 1125. Smallest Sufficient Team
 * https://leetcode.com/problems/smallest-sufficient-team/
 *
 * In a project, you have a list of required skills req_skills, and a list of people.
 * The i-th person people[i] contains a list of skills that person has.
 */

```

```

* Consider a sufficient team: a set of people such that for every required skill,
* there is at least one person in the team who has that skill.
* Return any sufficient team of the smallest possible size.
*
* Example 1:
* Input: req_skills = ["java","nodejs","reactjs"],
*        people = [["java"],["nodejs"],["nodejs","reactjs"]]
* Output: [0,2]
*
* Example 2:
* Input: req_skills = ["algorithms","math","java","reactjs","csharp","aws"],
*        people = [["algorithms","math","java"],["algorithms","math","reactjs"],
*                  ["java","csharp","aws"],["reactjs","csharp"],
*                  ["csharp","math"],["aws","java"]]
* Output: [1,2]
*
* Constraints:
* 1 <= req_skills.length <= 16
* 1 <= people.length <= 60
* 1 <= people[i].length <= 16
*
* ---
* Approach: Bitmask DP
*
* Since req_skills.length <= 16, we represent each skill set as a bitmask.
* dp[mask] = smallest list of people whose combined skills equal this mask.
*
* For each person, we iterate over all existing reachable masks and try
* combining the person's skill mask. If the combined mask yields a smaller
* team than currently known, we update it.
*
* Time:  $O(2^n * p)$  where  $n$  = number of skills,  $p$  = number of people.
* Space:  $O(2^n)$  for the dp map.
*/
public class SmallestSufficientTeam {

    /**
     * STATE: dp[mask] = The smallest list of people indices covering these skills.
     * BASE CASE: dp[0] = empty list (no skills, no people).
     * TRANSITION: If (existing team + this person) is smaller than current
     * dp[nextMask], update.
     */
    public int[] smallestSufficientTeam(String[] req_skills, List<List<String>> people) {
        int n = req_skills.length;

        // Map the req_skills to integer
        Map<String, Integer> skillsMap = new HashMap<>();
        for (int i = 0; i < n; i++) {
            skillsMap.put(req_skills[i], i);
        }

        Map<Integer, List<Integer>> dp = new HashMap<>();
        dp.put(0, new ArrayList<>());

        for (int i = 0; i < people.size(); i++) {

            int peopleSkillMask = 0;
            List<String> curr = people.get(i);
            for (int j = 0; j < curr.size(); j++) {
                String currentSkill = curr.get(j);
                peopleSkillMask |= (1 << skillsMap.get(currentSkill));
            }

            if (peopleSkillMask == 0) {
                continue;
            }

            for (Map.Entry<Integer, List<Integer>> entry : new HashMap<>(dp).entrySet()) {
                int combinedMask = entry.getKey() | peopleSkillMask;

```

```

        // If we found a new skill combination OR a shorter path to an existing
        // combination
        if (!dp.containsKey(combinedMask) || dp.get(combinedMask).size() > entry.getValue().size() + 1) {
            List<Integer> newTeam = new ArrayList<>(entry.getValue());
            newTeam.add(i);
            dp.put(combinedMask, newTeam);
        }
    }
}

// The answer is the team associated with the "all bits set" mask
List<Integer> resultList = dp.get((1 << n) - 1);
int[] result = new int[resultList.size()];
for (int i = 0; i < resultList.size(); i++) {
    result[i] = resultList.get(i);
}

return result;
}

public static void main(String[] args) {
    SmallestSufficientTeam solver = new SmallestSufficientTeam();
    String[] skills = { "java", "nodejs", "reactjs" };
    List<List<String>> people = Arrays.asList(
        Arrays.asList("java"),
        Arrays.asList("nodejs"),
        Arrays.asList("nodejs", "reactjs"));
    System.out.println("Smallest Team Indices: " + Arrays.toString(solver.smallestSufficientTeam(skills, people)));
}
}

```

Tree DP

337. House Robber III

```
package leetcode.dp.trees;

import java.util.HashMap;
import java.util.Map;

/**
 * 337. House Robber III
 * https://leetcode.com/problems/house-robber-iii/
 *
 * The thief has found a new place to rob. The houses form a binary tree.
 * The thief cannot rob two directly linked houses (parent-child).
 * Return the maximum amount of money the thief can rob.
 *
 * Example 1:
 *      3
 *     / \
 *    2   3
 *   \   \
 *    3   1
 * Output: 7 (3 + 3 + 1 = 7, rob root + grandchildren)
 *
 * Example 2:
 *      3
 *     / \
 *    4   5
 *   / \   \
 *  1  3  1
 * Output: 9 (4 + 5 = 9, rob the two children)
 *
 * Constraints:
 * The number of nodes is in the range [1, 10^4].
 * 0 <= Node.val <= 10^4
 *
 * ---
 * Approach: Tree DP (post-order DFS)
 *
 * For each node, return a pair [robThis, skipThis]:
 * robThis: max money if we ROB this node (can't rob children)
 * skipThis: max money if we SKIP this node (children free to be robbed or not)
 *
 * TRANSITION:
 * robThis = node.val + left[skipThis] + right[skipThis]
 * skipThis = max(left[robThis], left[skipThis]) + max(right[robThis], right[skipThis])
 *
 * ANSWER: max(root[robThis], root[skipThis])
 *
 * Time: O(n) -- visit each node once
 * Space: O(h) -- recursion stack, h = tree height
 */
public class HouseRobber3 {

    // Simple TreeNode definition for this problem
    static class TreeNode {
        int val;
        TreeNode left;
        TreeNode right;

        TreeNode(int val) {
            this.val = val;
        }
    }
}
```

```

/**
 * Optimal approach: return int[2] = {robThis, skipThis} for each node.
 * No extra hash map needed.
 */
public static int rob(TreeNode root) {
    int[] result = dfs(root);
    return Math.max(result[0], result[1]);
}

/**
 * Post-order DFS: returns [robThisNode, skipThisNode]
 */
private static int[] dfs(TreeNode node) {
    if (node == null) return new int[]{0, 0};

    int[] left = dfs(node.left);
    int[] right = dfs(node.right);

    // Rob this node: can't rob children, take their skip values
    int robThis = node.val + left[1] + right[1];

    // Skip this node: each child can be robbed or skipped (take max of each)
    int skipThis = Math.max(left[0], left[1]) + Math.max(right[0], right[1]);

    return new int[]{robThis, skipThis};
}

/**
 * Alternative: HashMap memoization (less optimal but intuitive)
 *
 * rob(node) = max money we can get from the subtree rooted at node.
 * Two choices:
 * 1. Rob node: node.val + rob(grandchildren)
 * 2. Skip node: rob(left child) + rob(right child)
 */
public static int robMemo(TreeNode root) {
    return robHelper(root, new HashMap<>());
}

private static int robHelper(TreeNode node, Map<TreeNode, Integer> memo) {
    if (node == null) return 0;
    if (memo.containsKey(node)) return memo.get(node);

    // Choice 1: Rob this node + skip children + rob grandchildren
    int robThis = node.val;
    if (node.left != null) {
        robThis += robHelper(node.left.left, memo) + robHelper(node.left.right, memo);
    }
    if (node.right != null) {
        robThis += robHelper(node.right.left, memo) + robHelper(node.right.right, memo);
    }

    // Choice 2: Skip this node, rob children
    int skipThis = robHelper(node.left, memo) + robHelper(node.right, memo);

    int result = Math.max(robThis, skipThis);
    memo.put(node, result);
    return result;
}

public static void main(String[] args) {
    // Example 1:
    //      3
    //     / \
    //    2  3
    //   \  \
    //    3  1
    TreeNode root1 = new TreeNode(3);
    root1.left = new TreeNode(2);
    root1.right = new TreeNode(3);
    root1.left.right = new TreeNode(3);
}

```

```

root1.right.right = new TreeNode(1);
System.out.println("Example 1 (pair): " + rob(root1)); // 7
System.out.println("Example 1 (memo): " + robMemo(root1)); // 7

// Example 2:      3
//                / \
//               4   5
//              / \  \
//             1  3  1
TreeNode root2 = new TreeNode(3);
root2.left = new TreeNode(4);
root2.right = new TreeNode(5);
root2.left.left = new TreeNode(1);
root2.left.right = new TreeNode(3);
root2.right.right = new TreeNode(1);
System.out.println("Example 2 (pair): " + rob(root2)); // 9
System.out.println("Example 2 (memo): " + robMemo(root2)); // 9
    }
}

```

3531. Maximum Good Subtree Score

```

package leetcode.dp.trees;

import java.util.*;

/**
 * 3531. Maximum Good Subtree Score
 * https://leetcode.com/problems/maximum-good-subtree-score/
 *
 * You are given an undirected tree rooted at node 0 with n nodes numbered
 * from 0 to n - 1; vals[i] is the value of the i-th node, and par[i] is
 * the parent of the i-th node (par[0] == -1 for the root).
 *
 * A subtree of node i is "good" if every digit (0-9) that appears in the
 * values of nodes in the subtree appears at most once. Return the maximum
 * sum of values in any good subtree. Since the answer may be large, return
 * it modulo 10^9 + 7.
 *
 * Constraints:
 * 1 <= n <= 10^4
 * 1 <= vals[i] <= 10^9
 *
 * ---
 * Approach: Tree DP with bitmask (digits 0-9 as 10-bit mask)
 *
 * Each digit 0-9 is mapped to a bit (10 bits total, 2^10 = 1024 states).
 * For a node's value, compute which digits appear and how many times.
 * If any digit repeats, that node alone is invalid (mask = -1).
 *
 * dp[mask] at each node = maximum sum achievable by selecting a subset of
 * nodes (from the subtree rooted at this node) whose combined digit-mask
 * is exactly `mask`. We merge children one by one using a knapsack-like
 * combination of non-overlapping masks.
 *
 * After processing all nodes, the answer is max(dp[mask]) at any node
 * for any mask, taken modulo 10^9 + 7.
 *
 * Time: O(n * 3^10) -- for each node, merging masks with each child is O(3^10)
 *        because we iterate over all pairs of submasks that don't overlap.
 * Space: O(n * 2^10) for storing dp at each node, simplified to O(2^10) per DFS frame.
 */
public class MaxGoodSubtreeScore {

    static final int MOD = 1_000_000_007;
    static final int FULL = 1 << 10; // 1024 states for 10 digits

```

```

public int goodSubtreeSum(int[] vals, int[] par) {
    int n = vals.length;

    // Build adjacency list (children only, since it's a rooted tree)
    List<List<Integer>> children = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        children.add(new ArrayList<>());
    }
    for (int i = 1; i < n; i++) {
        children.get(par[i]).add(i);
    }

    // Precompute the digit bitmask for each node's value.
    // digitMask[i] = bitmask of digits in vals[i], or -1 if any digit repeats.
    int[] digitMask = new int[n];
    for (int i = 0; i < n; i++) {
        digitMask[i] = computeDigitMask(vals[i]);
    }

    // DFS iteratively (to avoid stack overflow on large trees)
    // dp[node][mask] = max sum of a "good" subset from node's subtree with digit-mask = mask
    // We use long to avoid overflow before taking mod at the end.
    // -1 means this mask state is unreachable.
    long[][] dp = new long[n][FULL];
    for (long[] row : dp) Arrays.fill(row, -1);

    // Topological order via iterative DFS (post-order)
    int[] order = new int[n];
    int idx = 0;
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(0);
    while (!stack.isEmpty()) {
        int node = stack.pop();
        order[idx++] = node;
        for (int child : children.get(node)) {
            stack.push(child);
        }
    }

    // Process nodes in reverse order (leaves first, root last)
    for (int i = n - 1; i >= 0; i--) {
        int node = order[i];

        // Base case: the empty subset (select no nodes) has sum 0 and mask 0
        dp[node][0] = 0;

        // If this node's value has no repeated digits, we can select it alone
        if (digitMask[node] != -1) {
            dp[node][digitMask[node]] = vals[node];
        }

        // Merge each child's dp into this node's dp (knapsack-style)
        for (int child : children.get(node)) {
            // We need to merge dp[node] with dp[child].
            // To avoid using a child's result twice, iterate over current dp[node]
            // and combine with dp[child] where masks don't overlap.

            // Take a snapshot of current dp[node] to avoid mid-iteration mutation issues
            long[] snapshot = dp[node].clone();

            // For each reachable mask in the child
            for (int cmask = 0; cmask < FULL; cmask++) {
                if (dp[child][cmask] < 0) continue; // unreachable in child

                // Combine with each reachable mask in current node's dp
                // that doesn't overlap with cmask.
                // Iterate over submasks of the complement of cmask.
                int complement = (FULL - 1) ^ cmask;
                for (int pmask = complement; pmask >= 0; pmask = (pmask - 1) & complement) {
                    if (snapshot[pmask] >= 0) {

```

```

        int combined = pmask | cmask;
        dp[node][combined] = Math.max(dp[node][combined],
            snapshot[pmask] + dp[child][cmask]);
    }
    if (pmask == 0) break; // important: avoid infinite loop when pmask wraps
}
}
}

// The answer is the max across ALL nodes' subtrees, for any non-zero mask.
// Each node is the root of its own subtree, so check dp[every node].
long ans = 0;
for (int node = 0; node < n; node++) {
    for (int mask = 1; mask < FULL; mask++) {
        if (dp[node][mask] > ans) {
            ans = dp[node][mask];
        }
    }
}

return (int) (ans % MOD);
}

/**
 * Computes the digit bitmask for a number.
 * Returns a bitmask where bit i is set if digit i appears in the number.
 * Returns -1 if any digit appears more than once (invalid for a "good" subtree).
 */
private int computeDigitMask(int num) {
    int mask = 0;
    while (num > 0) {
        int digit = num % 10;
        int bit = 1 << digit;
        if ((mask & bit) != 0) {
            return -1; // Repeated digit
        }
        mask |= bit;
        num /= 10;
    }
    return mask;
}

public static void main(String[] args) {
    MaxGoodSubtreeScore solver = new MaxGoodSubtreeScore();

    // Test Case 1: vals = [2,3], par = [-1,0]
    // Node 0 (val=2), Node 1 (val=3, child of 0)
    // Best good subtree: {0,1} with sum = 5, digits {2,3} no repeats
    int[] vals1 = {2, 3};
    int[] par1 = {-1, 0};
    System.out.println("Test 1: " + solver.goodSubtreeSum(vals1, par1)); // Expected: 5

    // Test Case 2: vals = [1,5,2], par = [-1,0,0]
    // Tree: 0 -> {1, 2}. vals = {1, 5, 2}
    // All digit masks: 1->{1}, 5->{5}, 2->{2}. No overlaps.
    // Best: pick all 3 nodes, sum = 8
    int[] vals2 = {1, 5, 2};
    int[] par2 = {-1, 0, 0};
    System.out.println("Test 2: " + solver.goodSubtreeSum(vals2, par2)); // Expected: 8

    // Test Case 3: vals = [11,22,33], par = [-1,0,0]
    // All nodes have repeated digits (11->digit 1 twice, etc.)
    // No single node is valid, so answer = 0
    int[] vals3 = {11, 22, 33};
    int[] par3 = {-1, 0, 0};
    System.out.println("Test 3: " + solver.goodSubtreeSum(vals3, par3)); // Expected: 0
}
}

```